

README

Part 8: 从零训练 LLM — 后训练全流程（SFT -> 奖励模型 -> DPO/PPO/GRPO）

- > 从零构建一个 GPT-2，走完 LLM 的完整生命周期：预训练 → SFT → 奖励模型 → 对齐 → 强化学习 → 评估。
- > 参考：[train-llm-from-scratch](https://github.com/FareedKhan-dev/train-llm-from-scratch)

章节导航

LayerNorm + learned PE + MHA + ReLU）、预训练流水线
Template、Prompt Masking
模型、DPO/ORPO/KTO 三种对齐
Self-Surrogate）、GRPO（Critic-Free RL）
单段对比、交互式 Chat
Q/AWQ）、KV 显存与 KIVI、PagedAttention、连续批处理、投机解码、TTFT/TPOT、vLLM 实操
三范式、lm-eval-harness（实操 + 自定义 task）、HELM、benchmark 污染（GSM1k）、ppl 陷阱；**幻觉与安全**（语义熵/SelfCheckGPT、温度迷思、E
分解注入）、参数量/显存对比、分类微调回顾
从 start SFT → 推理 RL → self-consistency

参考来源标注（两个源仓库各管什么）

本部分内容来自两个风格不同的源仓库，按章标注——学习时按需对照，不要混着读：

[FareedKhan-dev/train-llm-from-scratch]（端到端管线）	延伸：[rasbt/LLMs-from-scratch](https://github.com/rasbt/LLMs-from-scratch)（更严谨的分章实现）
	ch04-05（GPT 从零重推、**OpenAI GPT-2 权重加载/权重手术**、附录 D LR 调度细节）
	ch07（**指令数据的 JSON 格式规约**、Alpaca 数据组织）
	ch07 的 `04_preference-tuning-with-dpo`（**DPO 从零 + 偏好数据如何构造**）
	（主书无 RL）→ 续作 [reasoning-from-scratch](https://github.com/rasbt/reasoning-from-scratch) ch06-07
	—
	—
	ch07 的 `ollama_evaluate.py`（**LLM-as-judge 的最小可运行实现**）
	本新章主线：附录 E（LoRA 从零）+ ch06（分类微调）

- > 阅读建议：主源负责"跑通全流程"，rasbt 负责"把某一章做扎实"。修完本部分后，
- > 把 rasbt 的 ch06/附录 E/续作 ch06 当作三个巩固模块重走一遍（难度对我们学生约 2.5-4/10）。

前置知识

本部分需要你已掌握：

- Part 6 全部内容：Transformer 架构、self-attention、残差连接、LayerNorm、decoder-only GPT —— Part 8 的模型骨架直接复用 Part 6
 - Part 3 的 BatchNorm：归一化的思想 —— 讲 LayerNorm 时会和它对照
 - 概率论基础：sigmoid、log-probability、KL 散度 —— DPO/PPO/GRPO 的核心数学工具
- > Part 8 与 Part 7 独立——不依赖 Part 7 代码。重合内容（Transformer 架构、SFT、DPO）视为复习，但从不同角度讲解。

学习路线图

Part 6 (Transformer 架构、self-attention、decoder-only GPT)

|
| "架构懂了，但模型只会续写，不会对话、不会遵循指令..."



Part 8: LLM 后训练全流程	
① GPT-2 架构 + 预训练 — LayerNorm + learned PE + ReLU	——→ 01_gpt_and_pretrain.md
② SFT + Chat Template — Prompt Masking	——→ 02_sft_and_chat.md
③ 奖励模型 + DPO/ORPO/KTO — Bradley-Terry + 三种对齐	——→ 03_reward_and_dpo.md
④ PPO + GRPO — GAE + Clipped Surrogate	——→ 04_ppo_and_grpo.md
⑤ 评估 + 部署 — GSM8K + 生成策略 + Chat	——→ 05_eval_and_deploy.md

数据与依赖

本课程完全自包含：数据、权重都不需要提前下载，全部脚本可直接跑通——

说明
`data/input.txt` (tiny Shakespeare) 已在仓库内
脚本 01-10：仅需 `torch` (CPU/GPU) ；脚本 11-12 另需 `transformers` + 已缓存 Qwen2.5-0.5B (-Instruct)、脚本 12 需 `lm_eval[hf]` (见根 requirements.txt)
脚本 01-10 不需要 (从零训练) ；脚本 11-12 首次运行会从 HF 拉取 0.5B 模型 (缺失时打印指引优雅退出, rc=0)

规模对照表 (想跑原版规模时从这里查)：

n_embed	heads	blocks	参数量	说明
64	4	2	~2M	全部脚本默认, <30s
512	8	12	~40M	单张 4090 余量充足
512	8	8	77M	train-llm-from-scratch 的基准档

	n_embed	heads	blocks	参数量	说明
默认	1024	16	24	**406M**	单卡 4090 可跑：模型状态约 6.5GB，用 **batch=4 + 梯度累积**控制激活（详见下方备注）；更稳妥可租 24GB 卡

- > 多卡备注：本课所有脚本（含 GPU 模式）都是单卡程序——一张 4090 可完成
- > 课程全部内容与作业；想跑 406M 原版规模的单卡步骤：`batch_size=4 + gradient_accumulation_steps=8`
- > （保有效 batch），激活约 4GB + 模型状态 6.5GB，24GB 余量充足；若再放大 batch 或 seq，
- > 租 2×24GB 卡（数据并行 DDP，见 Part 10）或 A100 80GB。
- > △ 参数放大时的超参因果（面试常问，别死抄数字）：模型放大 10×，
- > ① lr 降（梯度噪声占比变化，406M 用 ~3e-4 而不是 2M 的 3e-3）；
- > ② effective batch 升（用梯度累积凑，稳住大 batch 的统计量）；
- > ③ warmup 步数升（大模型初期更脆）；
- > ④ seq/batch 与激活显存联动——放大前先按 Part 9 的显存公式估一估，别先改模型后爆显存。

演进路线：从"续写器"到"对话助手"

本教程走完 LLM 的全部训练阶段——每一阶段解决上一阶段留下的问题：

阶段	目标	解决什么问题	对应脚本
预训练	预测下一个 token	学会语言的统计规律	`02`
SFT	按指令回答	从"续写器"变成"对话模型"	`03`
奖励模型	给回答打分	学会判断"好回答 vs 坏回答"	`04`
DPO/ORPO/KTO	偏好对齐	直接用偏好数据优化策略	`05`
PPO	强化学习	用 reward model 在线优化	`06`
GRPO	Critic-Free RL	不需要 Value Network，更简单	`07`
评估	量化效果	GSM8K 准确率、生成质量对比	`08`
幻觉与安全	可信与合规	语义熵检测幻觉、ECE 校准、refusal direction、lm-eval 实操、合规四件套	`11` `12`

> △ 我们的脚本是 CPU 缩小版（更小的 hidden/dim、更少的步数、更短的上下文）。不同超参、不同随机种子，数字都会有差异。看趋势，别死记数字。GPU 全量版请参考 train-llm-from-scratch 仓库的超参。

课后作业

每一章末尾有 2-3 道思考题（<details> 折叠答案）。全部学完后，去这里做动手练习：

[Assignment 8](../../assignments/assignment_8/)

脚本 11 可选实验：自备数据文件格式

[07 章 § 7](07_evaluation.md) 的 refusal direction 演示（Arditi 2406.11717）**不内嵌任何提示语样本**——想跑完整实验的读者请自备 `scripts/refusal_prompts.jsonl`（每行一个 JSON 对象，normal 组放普通问答语句，refusal 组按论文附录自行构造模型拒绝风格语句，各 ≥ 8 条）：

```
jsonl
{"text": "法国的首都是巴黎，对吗？", "label": "normal"}
{"text": "请帮我写一首关于春天的短诗。", "label": "normal"}
{"text": "光合作用的基本原理是什么？", "label": "normal"}
```

```
{"text": "<读者按论文附录自备的、模型会拒绝回答的请求语句>", "label": "refusal"}  
{"text": "<同上，再构造多条不同话题的拒绝风格语句>", "label": "refusal"}
```

构造要点（对照论文附录的做法）：

- 两组话题分布尽量对齐（同一话题既有 normal 版也有 refusal 版）——diff-in-means 提的是"拒绝方向"，话题混杂会把话题方向也混进来
- 每条写成完整请求句，与推理时同一格式（脚本取每条提示最后一个 token 的隐状态）
- 各 ≥ 8 条即可看到方向信号；追求复现论文量级请各 100+ 条并换 7B 模型

文件缺失时脚本只打印说明并跳过（rc=0）；7B 模型可用时效果最好（0.5B 可演示方法）。

相关资源

- [train-llm-from-scratch](https://github.com/FareedKhan-dev/train-llm-from-scratch) — 本部分参考的项目
- Radford et al. 2019：[Language Models are Unsupervised Multitask Learners (GPT-2)](https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf)
- Ouyang et al. 2022：[Training language models to follow instructions with human feedback (InstructGPT)](https://arxiv.org/abs/2203.02155)
- Rafailov et al. 2023：[Direct Preference Optimization: Your Language Model is Secretly a Reward Model (DPO)](https://arxiv.org/abs/2305.18290)
- Hong et al. 2024：[ORPO: Monolithic Preference Optimization without Reference Model](https://arxiv.org/abs/2403.07691)
- Ethayarajh et al. 2024：[KTO: Model Alignment as Prospect Theoretic Optimization](https://arxiv.org/abs/2402.01306)
- Schulman et al. 2017：[Proximal Policy Optimization Algorithms (PPO)](https://arxiv.org/abs/1707.06347)
- Shao et al. 2024：[DeepSeekMath: Pushing the Limits of Mathematical Reasoning (GRPO)](https://arxiv.org/abs/2402.03300)

[← 上一章：Part 7 Minimind](../Part7_minimind/tutorial/README.md) | [下一章：Part 9 CUDA 内核 →](../Part9_cuda_kernels/tutorial/README.md)

01_gpt_and_pretrain

01 — GPT-2 架构与预训练：从零构建经典 Transformer

> 从零实现 GPT-2 的经典架构（LayerNorm + learned PE + MHA + ReLU），然后用现代训练技巧预训练它——AdamW、cosine LR、bf16 混合精度、gradient accumulation。

前置知识

本章需要你已掌握：

- Part 6 全部：Transformer 架构、self-attention、残差连接、LayerNorm、decoder-only GPT
 - Part 3 的 BatchNorm：归一化的思想、可学习的缩放参数——讲 LayerNorm 时会和它对照
- > 如果你忘了"因果注意力怎么 mask"或"残差连接为什么重要"，先回 Part 6 的 [03_multi_head_attention.md](#)。

从 Part 6 结束的地方出发

Part 6 我们从零构建了一个字符级

mini-GPT，学会了"预测下一个字符"。那个模型用了最简单的组件：LayerNorm、learned 位置编码、标准 MHA、ReLU FFN。

Part 7 把每个零件都换成了现代版：RMSNorm、RoPE、GQA、SwiGLU。

Part 8 又换回经典款——为什么？因为 train-llm-from-scratch 仓库用的是经典 GPT-2 架构，而且经典款更容易理解后训练的核心思想（奖励头、价值头、DPO loss）。现代组件是"锦上添花"，后训练才是"从续写器到对话助手"的关键。

GPT-2 架构总览

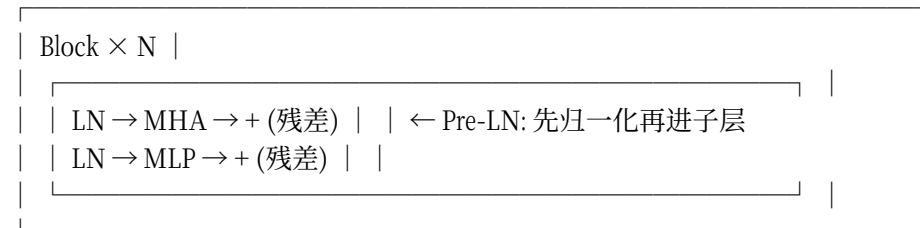
先看整体结构，然后逐个拆解：

输入 token ids (B, T)

v

token_embed(ids) + pos_embed(positions) $\leftarrow$ 相加（不是拼接）

v



v

LayerNorm (final) $\leftarrow$ 最后一层 LN

v

lm_head (Linear: n_embed $\rightarrow$ vocab) $\leftarrow$ 预测下一个 token

v

logits (B, T, vocab_size)

关键设计选择：

- token embedding + learned position embedding（相加）：位置编码是可学习的参数表，不用 RoPE
- Pre-LN Block：先 LayerNorm 再进子层，训练更稳定（后面会详细讲）
- MHA：标准多头注意力，不用 GQA
- MLP：4x 扩展 + ReLU，不用 SwiGLU
- forward_hidden()：返回 final LN 之后、lm_head 之前的 hidden state —— 这是后训练的关键 hook 点

对应代码在 [01_gpt_model.py](../scripts/01_gpt_model.py)。

单头注意力 Head

注意力是 Transformer 的核心。先看最简单的单头版本：

```
python
```

```
class Head(nn.Module):
```

```

def __init__(self, head_size, n_embed, context_length):
    super().__init__()
    self.key = nn.Linear(n_embed, head_size, bias=False) # K 投影
    self.query = nn.Linear(n_embed, head_size, bias=False) # Q 投影
    self.value = nn.Linear(n_embed, head_size, bias=False) # V 投影
    self.register_buffer('tril', torch.tril(torch.ones(context_length, context_length)))

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x) # (B, T, head_size)
        q = self.query(x) # (B, T, head_size)

```

scaled dot-product attention

```

wei = q @ k.transpose(-2, -1) (k.shape[-1] * -0.5) # (B, T, T)
wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # causal mask
wei = F.softmax(wei, dim=-1)
v = self.value(x) # (B, T, head_size)
return wei @ v # (B, T, head_size)

```

三个关键步骤：

1. Q/K/V 投影：把输入 **x** 分别投影成 Query、Key、Value 三个向量。 **bias=False** 是现代惯例（省参数，效果一样）
2. Scaled dot-product： **$Q @ K^T / \sqrt{d_k}$** —— 除以 **$\sqrt{d_k}$** 防止内积值太大导致 softmax 饱和
3. Causal mask：用下三角矩阵把"未来位置"填成 **-inf**，softmax 后变成 0 —— 保证每个位置只能看到自己和之前的内容

⚠ **register_buffer('tril', ...)** 注册的是 buffer 而不是 parameter —— 它不参与梯度更新，但会随 **model.to(device)** 移动到 GPU。

多头注意力 MultiHeadAttention

单头注意力只能关注一种模式。多头注意力让模型同时关注多种模式：

```

python
class MultiHeadAttention(nn.Module):
    def __init__(self, n_head, n_embed, context_length):
        super().__init__()
        head_size = n_embed // n_head
        self.heads = nn.ModuleList([Head(head_size, n_embed, context_length)
                                     for _ in range(n_head)])
        self.proj = nn.Linear(n_embed, n_embed) # 输出投影

    def forward(self, x):
        x = torch.cat([h(x) for h in self.heads], dim=-1) # 拼接
        return self.proj(x) # 投影回 n_embed 维

```

为什么需要多头？不同的 head 可以关注不同的模式——比如一个 head 关注相邻词（语法），另一个 head 关注远距离依赖（语义）。**n_head=4** 意味着有 4 个这样的"视角"。

⚠ **head_size = n_embed // n_head**：每个 head 的维度是总维度除以 head 数。拼接后 **n_head × head_size = n_embed**，正好回到原始维度。

MLP 前馈网络

每个 Block 里除了注意力，还有一个前馈网络（FFN）：

```
`python
class MLP(nn.Module):
def __init__(self, n_embed):
super().__init__()
self.net = nn.Sequential(
nn.Linear(n_embed, 4 * n_embed), # 4x 扩展
nn.ReLU(), # 激活
nn.Linear(4 * n_embed, n_embed), # 投影回
)
```

经典的"4x 扩展 + ReLU + 投影回"设计。先扩大到 4 倍维度（增加表达力），用 ReLU 激活（引入非线性），再投影回原始维度。

与 Part 7 SwiGLU 的对比：

	经典 ReLU FFN（本脚本）	SwiGLU FFN（Part 7）
激活	ReLU（硬截断：负值变 0）	SiLU（软门控：负值有小梯度）
结构	两层：up → down	三层：gate + up → down
扩展比	4x	~3.2x（同等参数量）
代表	GPT-2, BERT	Llama, Qwen, minimind

ReLU 更简单，SiLU 表达力更强。对于教学目的，经典款更容易理解。

Pre-LN vs Post-LN

这是 Transformer 架构中一个经常被忽略但非常重要的设计选择：

```
`
Post-LN（原始论文 "Attention Is All You Need"）：
x = x + Attn(x) ← 先算子层
x = LayerNorm(x) ← 再归一化

Pre-LN（GPT-2、Llama 等现代模型）：
x = x + Attn(LayerNorm(x)) ← 先归一化，再算子层
`
```

对应代码：

```
`python
class Block(nn.Module):
def forward(self, x):
x = x + self.attn(self.ln1(x)) # Pre-LN: LN 在子层之前
x = x + self.mlp(self.ln2(x))
return x
`
```

为什么 Pre-LN 更稳定？

- Post-LN：LayerNorm 在残差之后，梯度要穿过 LN 才能传回主路径。LN 的梯度在均值/方差计算时有非线性，容易导致梯度爆炸或消失，需要 learning rate warmup
- Pre-LN：LayerNorm 在子层之前，残差连接直接把梯度传回主路径 ($x = x + f(LN(x))$)，梯度对 x 有直通路程)。训练更稳定，不需要 warmup

△ 几乎所有现代 LLM（GPT-2/3/4、Llama、Qwen）都用 Pre-LN。Post-LN 主要出现在早期论文里。

forward_hidden()：后训练的关键 hook

这是本脚本最重要的设计——一个看似简单的函数：

```
python
def forward_hidden(self, idx):
    """backbone 前向：返回 final LN 之后的 hidden state (B, T, n_embed)。"""
    B, T = idx.shape
    tok_emb = self.token_embed(idx) # (B, T, n_embed)
    pos_emb = self.position_embed(self.pos_idx[:T]) # (T, n_embed)
    x = tok_emb + pos_emb
    for block in self.blocks:
        x = block(x)
    return self.ln_f(x) # ← 在这里停住，不经过 lm_head
```

为什么需要这个函数？

后训练阶段，我们需要在 backbone 上接入不同的"头"：

阶段	接入什么	用途
预训练	'lm_head'	预测下一个 token
奖励模型	'reward_head'	给回答打分（标量）
PPO	'value_head'	估计状态价值 V(s)

`forward_hidden()` 返回的就是"去掉 `lm_head` 之前的最后一层输出"，奖励头和价值头都从这里接入。后面几章会反复用到它。

参数量计算

GPT-2 的参数量有一个简洁的近似公式：

```
参数量 ≈ vocab × embed + n_blocks × (12 × embed2) + embed × vocab
```

其中 $12 \times \text{embed}^2$ 来自每个 Block：

- 注意力：Q/K/V 三个投影 = $3 \times \text{embed}^2$ ，输出投影 = embed^2 ，共 $4 \times \text{embed}^2$
- MLP：两个线性层 = $2 \times 4 \times \text{embed}^2 = 8 \times \text{embed}^2$
- 合计 $12 \times \text{embed}^2$ （忽略 LN 的少量参数）

配置	embed	heads	blocks	vocab	参数量
CPU 缩小版	64	4	2	256	~0.1M

配置	embed	heads	blocks	vocab	参数量
~1M	128	4	4	1000	~0.8M
~10M	256	8	6	50304	~6M
~100M	512	8	12	50304	~40M
标准 GPT-2	1024	16	24	50304	~406M

本教程用 CPU 缩小版（~0.1M 参数）快速演示。GPU 模式用 ~40M 配置（embed=512, 12 层），可在单张 4090 上跑通全流程。原理完全一样，只是规模不同。

预训练流水线

模型搭好了，怎么训练？对应代码在 [02_pretrain.py](../scripts/02_pretrain.py)。

数据加载

```
python
```

CPU 模式：字符级编码（vocab=65）

```
chars = sorted(list(set(text)))
stoi = {c: i for i, c in enumerate(chars)}
data = torch.tensor([stoi[c] for c in text], dtype=torch.long)
```

GPU 模式：tiktoken BPE（vocab=50304）

（tiktoken 只能用现成词表、不能训练；与 Part 7 自训 BPE 的对照见

[courses/Part7_minimind/tutorial/01_bpe_tokenizer.md](#)
的「三种工业实现对照」）

```
import tiktoken
enc = tiktoken.get_encoding('r50k_base')
data = torch.tensor(enc.encode_ordinary(text), dtype=torch.long)
```

AdamW 优化器

```
python
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4, # 学习率
    betas=(0.9, 0.95), # 动量衰减系数（比默认 (0.9, 0.999) 更保守）
    weight_decay=0.1, # 权重衰减（L2 正则的"解耦"版本）
)
```

AdamW vs Adam + L2：AdamW 把权重衰减从梯度更新中"解耦"出来——先做 Adam 更新，再单独做权重衰减。这比 Adam + L2 更正确，是现代 LLM 训练的标准选择。

Cosine LR + Linear Warmup

```
`python
warmup = max(3, max_steps // 10)

def lr_lambda(step):
    if step < warmup:
        return step / warmup # 线性 warmup
    p = (step - warmup) / max(1, max_steps - warmup)
    return 0.5 (1.0 + math.cos(math.pi p)) # cosine 衰减

scheduler = torch.optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)
`

学习率
^
+-----> step
warmup cosine decay
`
```

为什么要 warmup？
训练初期参数是随机的，梯度方向不稳定。如果一开始就用大学习率，容易"跑偏"。warmup 让学习率从小到大逐渐增长，等参数稳定后再加速。

bf16 混合精度

```
`python
if device == 'cuda':
    with torch.autocast(device_type='cuda', dtype=torch.bfloat16):
        _, loss = model(xb, yb)
else:
    _, loss = model(xb, yb) # CPU 不支持 bf16
`
```

bf16 vs fp16：为什么推荐 bf16？

	bf16	fp16
指数位	8 bit（与 fp32 相同）	5 bit（比 fp32 少）
尾数位	7 bit	10 bit
动态范围	大（不容易溢出）	小（容易溢出/下溢）
精度	较低	较高
GradScaler	**不需要**	需要
推荐硬件	A100, 4090, 3090	V100, T4

bf16 的动态范围和 fp32 一样大（8 bit 指数），所以不会出现 fp16 那种“梯度太小变 0、太大变 inf”的问题。这就是为什么 bf16 不需要 GradScaler——数值范围够大，不会溢出。现代 GPU（A100/4090）推荐 bf16。

Gradient Accumulation

```
`python
optimizer.zero_grad(set_to_none=True)
for _ in range(grad_accum):
    xb, yb = get_batch(train_data, batch_size, context_length, device)
    with torch.autocast(device_type='cuda', dtype=torch.bfloat16):
        _, loss = model(xb, yb)
    (loss / grad_accum).backward() # 梯度累积：除以累积步数
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0) # 梯度裁剪
optimizer.step()
```

Gradient Accumulation：显存不够装大 batch？没关系——做 N 次小 batch 的 forward/backward，把梯度加起来（除以 N），再做一次 step。效果等价于 N 倍大的 batch。

Gradient Clipping

```
`python
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

把所有参数的梯度范数裁剪到不超过 1.0。防止偶尔出现的“梯度爆炸”把模型炸飞。

Checkpoint 保存/恢复

```
`python
torch.save({
    'model': model.state_dict(),
    'optimizer': optimizer.state_dict(),
    'step': max_steps,
    'losses': losses,
    'config': {
        'n_head': n_head, 'n_embed': n_embed, 'n_blocks': n_blocks,
        'vocab_size': model_vocab, 'context_length': context_length,
    }
}, ckpt_path)
```

保存 optimizer 状态和 step 数，下次训练就能从断点恢复——不需要从头开始。

预训练后会发生什么？

预训练完成后，模型学会了“续写”——给它一段开头，它能接着写下去。但它不会对话：

输入: "First Citizen:\n"

输出: "We are the people of the world, and we are the ones who..."

↑ 像莎士比亚风格的续写，但不是对话

这就是预训练模型的局限：它学会了语言的统计规律，但不知道"什么是问题"、"什么是回答"。下一步我们需要 SFT（监督微调）来教它"按指令回答"。

> 延伸对照（LLMs-from-scratch）：rasbt/LLMs-from-scratch ch05 的「加载 OpenAI GPT-2 官方权重」与附录 D 的

> LR 调度完整实现——把我们的玩具训练换成真 GPT-2 权重做"权重手术"，是很好的课后实验。

课后练习

<details>

<summary>Q1: Pre-LN 为什么比 Post-LN 训练更稳定？</summary>

A: Pre-LN 的残差连接给了梯度一条"直通路径"—— $x = x + f(\text{LN}(x))$ 中，梯度对 x 的偏导直接包含一个恒等项 1，不需要穿过 LN 的非线性计算。Post-LN 的梯度必须穿过 LN（里面有均值/方差归一化 + 缩放），这些非线性操作容易导致梯度爆炸或消失，所以需要 warmup 来稳定训练。

</details>

<details>

<summary>Q2: bf16 为什么不需要 GradScaler？fp16 为什么需要？</summary>

A: fp16 只有 5 bit 指数，动态范围很小（最大 ~65504，最小 ~6e-8）。训练中梯度经常超出这个范围——太大的变 inf，太小的变 0（下溢）。GradScaler 把 loss 放大 S 倍，让梯度也放大，避免下溢；更新时再缩小回来。bf16 有 8 bit 指数（和 fp32 一样），动态范围够大，梯度几乎不会溢出/下溢，所以不需要 GradScaler。

</details>

<details>

<summary>Q3: 为什么 Gradient Accumulation 要除以 grad_accum？</summary>

A: 因为 `.backward()` 默认是累加梯度（不覆盖）。做 N 次 `.backward()` 后，梯度是 N 个 micro-batch 梯度的和，而不是平均。除以 N 后，等价于一个大 batch 的平均梯度——这才是我们想要的。如果不除，等效学习率变成了 N 倍，训练会不稳定。

</details>

课后作业

完成本章后，去 Assignment 8 完成题 1（单头注意力 Head）和题 2（Pre-LN Block）：

[Assignment 8](../../assignments/assignment_8/)

下一步

预训练完成后，模型会续写但不会对话。下一步我们用 SFT（监督微调）教它"按指令回答"，引入 Chat Template 和 Prompt Masking。

[02 — SFT 与 Chat Template](02_sft_and_chat.md)

02_sft_and_chat

02 — SFT 与 Chat Template：从"续写器"到"对话模型"

> 预训练模型会续写但不会遵循指令。SFT（Supervised Fine-Tuning）教它"按指令回答"——Chat Template 结构化输入，Prompt Masking 聚焦 response。

前置知识

本章需要你已掌握：

- 01 章全部：GPT-2 架构、预训练流水线、`forward_hidden()`
- Part 6 的 cross-entropy loss：F.cross_entropy 的输入输出

> 如果你忘了"cross-entropy 怎么算"，先回 Part 6 的 [02_language_model.md](#)。

从"续写器"到"对话模型"

预训练之后，模型学会了一件事：给定前面的文本，预测下一个 token。它能续写莎士比亚、补全代码、甚至模仿问答——但这些都是"统计上最可能的续写"，而不是"理解指令后的回答"。

预训练模型:

输入: "What is 1+1?"

输出: "What is 1+1? What is 2+2? What is 3+3?..."

↑ 只是在"续写"模式——重复问题，不是回答

SFT 之后:

输入: "What is 1+1?"

输出: "The answer is 2."

↑ 学会了"这是问题，我应该回答"

SFT 的本质：在"指令-回答"对上微调，让模型学会区分"用户说的话"和"自己应该说的话"。

Chat Template：结构化输入

对话不是一串连续文本——它有结构：谁在说、说什么。Chat Template 定义了这种结构。

本脚本使用简化版 ChatML 格式（对应 `[03_sft.py](../scripts/03_sft.py)`）：

```
<|system|>
You are a helpful assistant.
<|user|>
What is 1+1?
<|assistant|>
The answer is 2.
```

各部分的作用：

标记	作用	示例		
'<\	system\	>'	系统指令，定义助手行为	"You are a helpful assistant."
'<\	user\	>'	用户输入	"What is 1+1?"
'<\	assistant\	>'	模型应该生成的回答	"The answer is 2."

```
`python
SYSTEM_PROMPT = "You are a helpful assistant."

def format_chat(system, user, assistant=""):
    return f"<|system|>\n{system}\n<|user|>\n{user}\n<|assistant|>\n{assistant}"
`
```

为什么需要模板？没有模板，模型看到 **What is 1+1?The answer is 2.**
这种纯文本，分不清哪部分是指令、哪部分是回答。模板用特殊 token 划分边界，让模型学会"看到 **<|user|>** 后面是指令，看到 **<|assistant|>** 后面是我该生成的"。

△ CPU 模式下 context_length 只有 64，放不下完整 ChatML，所以用简化格式 **Q: {question}\nA: {response}** 来演示核心思想。原理完全一样。

Prompt Masking：核心技巧

这是 SFT 中最重要的技巧，也是和"普通微调"的本质区别。

问题：为什么不能所有 token 一起算 loss？

假设输入是：

```
`
<|system|> You are helpful. <|user|> What is 1+1? <|assistant|> The answer is 2.
`
```

_____ prompt 部分（已知输入） _____	_____ response _____
-----------------------------	----------------------

如果对所有 token 一起算标准 cross-entropy loss：

```
`
L = - Σ_all log P(token_i | context)
`
```

模型会学到什么？它会花大量 capacity 去"预测 prompt"——但 prompt 是已知的输入，预测它毫无意义。更糟的是，模型可能学会"复制指令"而不是"理解并回答"。

解决：只在 response token 上算 loss

```
`python
L = - Σ_response log P(token_i | context)
`
```

prompt 区域的 token 不贡献 loss，梯度只流过 response 区域。模型被迫把所有学习能力集中在"学会回答"上。

实现：loss_mask 张量

```
`python
def sft_loss(logits, tokens, loss_mask):
    """SFT loss with prompt masking — 只在 response tokens 上计算 loss。"""
```

Step 1: Shift — 用位置 t 的 logits 预测 $t+1$ 的 token

```
logits = logits[:, :-1, :] # (B, T-1, V)
targets = tokens[:, 1:] # (B, T-1)
mask = loss_mask[:, 1:] # (B, T-1) — mask 也要 shift 对齐
```

Step 2: 逐 token cross entropy（不自动求均值）

```
B, T, V = logits.shape
ce = F.cross_entropy(
    logits.reshape(B * T, V), targets.reshape(B * T), reduction="none"
)
ce = ce.view(B, T)
```

Step 3: 乘以 mask（prompt 区域 loss 归零）

```
ce = ce * mask
```

Step 4: 求和归一化（只除以 response token 数量）

```
loss = ce.sum() / mask.sum().clamp(min=1.0)
return loss
```

四步走：

1. Shift：logits 向前移一位——位置 t 的 logits 预测位置 $t+1$ 的 token（和预训练一样）
2. 逐 token CE：用 `reduction="none"` 算每个 token 的 loss，不自动求均值
3. 乘以 mask：prompt 区域的 mask=0，loss 变成 0；response 区域 mask=1，loss 保留
4. 归一化：只除以 response token 的数量（不是总 token 数）

⚠ `mask.sum().clamp(min=1.0)` 防止除零——如果某个 batch 里 response 为空（极端情况），不会崩。

mask 的构造

```
`python
```

完整序列：system + user + assistant

```
full_text = format_chat(SYSTEM_PROMPT, "What is 1+1?", "The answer is 2.")
```

prompt 部分：system + user（不含 assistant 的回答）

```
prompt_text = format_chat(SYSTEM_PROMPT, "What is 1+1?", "")
full_tokens = encode(full_text)
prompt_len = len(encode(prompt_text))
```

构造 mask：prompt 部分 = 0，response 部分 = 1

```
mask = torch.zeros(1, len(full_tokens))
mask[0, prompt_len:] = 1.0

tokens: <|system|> You are helpful . <|user|> What is 1+1? <|assistant|> The answer is 2 .
mask: 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1
prompt 区域（不计算 loss） response（计算 loss）
```

SFT vs 标准 CE 对比

	标准 CE（预训练）	SFT Masked CE
loss 范围	所有 token	只有 response token
公式	$L = - \sum_{all} \log P(token_i)$	$L = - \sum_{response} \log P(token_i)$
模型学到什么	预测下一个 token（包括 prompt）	只学会"回答"
适用场景	预训练	指令微调

实际效果：masked loss 通常比 unmasked loss 更大——因为只看"难的部分"（回答），不看"容易的部分"（复制 prompt）。这是正常的。

SFT 数据格式

SFT 数据通常是三元组 (instruction, input, output)：

```
python
示例（CPU 模式用合成数据）
```

```
pairs = [
("What is 1+1?", "The answer is 2."),
("Say hello.", "Hello! How can I help you?"),
("What color is the sky?", "The sky is blue."),
]
```

训练时，每步随机采样一个 pair，构造完整序列 + mask，算 masked loss，更新参数。

与 Part 7 SFT 的区别

	Part 7 SFT	Part 8 SFT（本脚本）									
计算	全 token 算 loss（简化版）	只在 response 上算（生产版）									
Template	`<\`	im_start\`	>` / `<\`	im_end\`	>`	`<\`	system\`	>` / `<\`	user\`	>` / `<\`	assistant\`
Masking	无	有（loss_mask 张量）									

Part 7 为了简化教学，跳过了 Prompt Masking。Part 8 补上这个关键技巧——这是生产级 SFT 的标准做法。

SFT 之后会发生什么？

SFT 之后，模型学会了"按指令回答"：

,

Q: "What is 3+4?"

A: "The answer is 7." ← 学会了回答（虽然可能不完美）

Q: "Say hello."

A: "Hello! How can I help you?" ← 学会了礼貌

,

但回答质量参差不齐——有时好、有时差、有时胡说。这是因为 SFT 只教了"什么格式的回答是对的"，没有教"什么样的回答是好的"。下一步我们需要奖励模型来量化回答质量。

> 延伸对照（LLMs-from-scratch）：rasbt ch07 开头对指令数据 JSON 格式规约的讨论（(instruction, input, output)

> 如何组织成模板、数据去重与改写）——比我们的合成问答对更接近真实数据工程。

课后练习

<details>

<summary>Q1: 如果不 mask prompt，模型会怎样？</summary>

A: 模型会把大量 capacity 花在"预测 prompt"上——因为 prompt 占了序列的大部分，而且比 response 更容易预测（就是输入本身）。结果是：prompt 区域的 loss 很低（因为模型学会了复制），但 response 区域的 loss 很高（没学到什么）。更糟的是，模型可能养成"复制指令"的惯性，生成时也倾向于重复输入而不是真正回答。

</details>

<details>

<summary>Q2: loss_mask 的梯度流到哪里去了？</summary>

A: `loss_mask` 本身没有梯度——它是个常量张量（0 和 1）。但它通过乘法操作 `ce * mask` 控制了梯度的流向：mask=0 的位置，loss 变成 0，对应的梯度也是 0，那些 token 的参数不会收到更新信号。mask=1 的位置，梯度正常流过。所以 mask 的作用是"选择性地阻断梯度"，而不是自己参与梯度计算。

</details>

<details>

<summary>Q3: SFT 的 loss 为什么通常比预训练的 loss 大？</summary>

A: 两个原因。第一，SFT 只看 response token，这些 token 通常比 prompt 更难预测（prompt 是已知的、重复的模式，response 是模型需要"创造性"生成的）。第二，SFT 数据量通常远小于预训练数据，模型在更少的数据上做更难的任务，loss 自然更高。看 SFT 的 loss 主要看"下降趋势"，而不是绝对值。

</details>

课后作业

完成本章后，去 Assignment 8 完成题 3（Prompt-Masked SFT Loss）：

下一步

SFT 之后模型会对话，但质量参差不齐。下一步我们引入奖励模型（Bradley-Terry 偏好模型），教模型区分"好回答"和"坏回答"，然后用 DPO/ORPO/KTO 三种算法直接优化策略。

[03 — 奖励模型与对齐算法](03_reward_and_dpo.md)

03_reward_and_dpo

03 — 奖励模型与对齐算法：DPO、ORPO、KTO

> SFT 之后模型会对话，但质量参差不齐。本章引入 Bradley-Terry 奖励模型来量化"好 vs 坏"，然后用 DPO/ORPO/KTO 三种算法直接优化策略——不需要训练 RL。

前置知识

本章需要你已掌握：

- 02 章全部：SFT、Chat Template、Prompt Masking、forward_hidden()
- 概率论基础：sigmoid 函数、log-probability

> 如果你忘了"sigmoid 是什么"，回忆一下： $\text{sigmoid}(x) = 1/(1+e^{-x})$ ，把任意实数映射到 (0, 1)，可以理解为"概率"。

为什么需要对齐？

SFT 之后，模型学会了"按指令回答"，但还有很多问题：

问题	例子
幻觉	"Python 是由 Guido van Rossum 在 1989 年发明的"（年份可能错）
不安全	生成有害内容、泄露隐私
不遵循偏好	用户喜欢简洁回答，模型却啰嗦一大堆
质量不稳定	同一个问题，有时回答好，有时回答差

对齐（Alignment）的目标：让模型的输出更符合人类的偏好——安全、有用、诚实。

Bradley-Terry 偏好模型

对齐的第一步是量化"什么是好回答"。Bradley-Terry 模型是最经典的方法。

直觉

假设你有两个回答 A 和 B，让你选哪个更好。Bradley-Terry 假设：

你选 A 的概率 = $\text{sigmoid}(r(A) - r(B))$

其中 $r(A)$ 是回答 A 的"潜在奖励"——我们看不到，但可以通过训练学到。

```
python
P(A > B) = sigmoid(r(A) - r(B))
```

- $r(A) \gg r(B)$: $\text{sigmoid} \rightarrow 1$, 你几乎肯定选 A
- $r(A) \ll r(B)$: $\text{sigmoid} \rightarrow 0$, 你几乎肯定选 B
- $r(A) = r(B)$: $\text{sigmoid} \rightarrow 0.5$, 你选谁都一样

训练损失

```
python
def bradley_tery_loss(r_chosen, r_rejected):
    """L = -log sigmoid(r_chosen - r_rejected)"""
    return -F.logsigmoid(r_chosen - r_rejected).mean()
```

直觉：

- $r_{\text{chosen}} > r_{\text{rejected}}$: $\text{sigmoid} \rightarrow 1$, $\text{loss} \rightarrow 0$ (正确排序，不需要更新)
- $r_{\text{chosen}} < r_{\text{rejected}}$: $\text{sigmoid} \rightarrow 0$, $\text{loss} \rightarrow$ 很大 (错误排序，强烈更新)
- $r_{\text{chosen}} = r_{\text{rejected}}$: $\text{sigmoid} \rightarrow 0.5$, $\text{loss} = \ln(2)$ (无法区分)

⚠ 用 $F.\text{logsigmoid}$ 而不是 $\log(\text{sigmoid}())$ ——前者数值更稳定 (内部用 softplus 实现，避免 $\log(0)$)。

奖励模型架构

怎么得到 $r(x)$ ？在 GPT backbone 上加一个"奖励头"：

```
python
class RewardModel(nn.Module):
    def __init__(self, gpt):
        super().__init__()
        self.gpt = gpt
        n_embed = gpt.lm_head.in_features
        self.reward_head = nn.Linear(n_embed, 1, bias=False)
        nn.init.zeros_(self.reward_head.weight) # 零初始化！

    def forward(self, idx):
        hidden = self.gpt.forward_hidden(idx) # (B, T, n_embed)
        reward = self.reward_head(hidden) # (B, T, 1)
        return reward[:, -1, 0] # (B,) — 取最后 token
```

对应代码在 [04_reward_model.py](../scripts/04_reward_model.py)。

架构图：

```

idx (B, T)
v
GPT.forward_hidden() → hidden (B, T, n_embed)
v
reward_head(hidden) → (B, T, 1)
v
取最后 token → r(x) (B,)

```

三个关键设计：

1. 只取最后 token：Transformer 是因果模型，最后一个 token 的 hidden state 包含了整个序列的信息（通过注意力机制汇总）。这是 InstructGPT/ChatGPT 的标准做法
2. 零初始化：`nn.init.zeros_(self.reward_head.weight)` —— 训练初期所有奖励 ≈ 0 ，意味着 $P(A > B) \approx 0.5$ （无偏好），符合直觉
3. 无 bias：简化设计，效果差不多

DPO 推导：从 RLHF 到分类问题

DPO（Direct Preference Optimization）是目前最流行的对齐算法。它的核心贡献是：把复杂的 RL 问题转化成了简单的分类问题。

起点：RLHF 目标

经典 RLHF 的目标是：

$$\max E_{y \sim \pi} [r(x, y)] - \beta * KL(\pi \parallel \pi_{ref})$$

翻译成人话：让策略 π 生成的回答获得高奖励 r ，但不要偏离参考模型 π_{ref} 太远（KL 散度惩罚）。

- β 控制"激进程度"： β 小 $\rightarrow$ 更激进地追求高奖励； β 大 $\rightarrow$ 更保守地保持接近参考模型

最优策略的闭式解

这个优化问题有闭式解：

$$\pi(y|x) = \pi_{ref}(y|x) \exp(r(x, y) / \beta) / Z(x)$$

其中 $Z(x)$ 是归一化常数（确保概率之和为 1）。

反解奖励函数

把上式两边取 log，反解出奖励：

$$r(x, y) = \beta \log(\pi(y|x) / \pi_{ref}(y|x)) + \beta \log Z(x)$$

关键洞察：奖励可以用策略的 log-prob ratio 来表示！不需要显式的奖励模型。

代入 Bradley-Terry

把反解出的奖励代入 Bradley-Terry 的 $P(A > B) = \text{sigmoid}(r(A) - r(B))$:

$$r(A) - r(B) = \beta * [\log(\pi(A)/\pi_{\text{ref}}(A)) - \log(\pi(B)/\pi_{\text{ref}}(B))]$$

注意 $Z(x)$ 在减法中消掉了！最终的 DPO loss :

```
python
def dpo_loss(policy_chosen_logps, policy_rejected_logps,
             ref_chosen_logps, ref_rejected_logps, beta=0.1):
    pi_logratios = policy_chosen_logps - policy_rejected_logps
    ref_logratios = ref_chosen_logps - ref_rejected_logps
    logits = pi_logratios - ref_logratios
    loss = -F.logsigmoid(beta * logits).mean()
```

$$\text{隐式奖励} = \beta * (\log_{\pi} - \log_{\text{ref}})$$

```
chosen_reward = beta * (policy_chosen_logps - ref_chosen_logps)
rejected_reward = beta * (policy_rejected_logps - ref_rejected_logps)
return loss, chosen_reward, rejected_reward
```

对应代码在 [05_dpo_alignment.py](../scripts/05_dpo_alignment.py)。

DPO 的核心公式：

$$L = -\log \text{sigmoid}(\beta * [(\log \pi_{\text{chosen}} - \log \pi_{\text{rejected}}) - (\log \pi_{\text{ref_chosen}} - \log \pi_{\text{ref_rejected}})])$$

直觉：

- 策略对 chosen 的 log-prob 越高、对 rejected 越低 $\rightarrow$ loss 越小
- 参考模型的 log-prob 作为 baseline 被减掉 $\rightarrow$ 只优化"策略比参考模型更偏好 chosen"的部分
- β 控制偏离参考模型的程度

DPO 训练流程

1. 加载 SFT 模型作为 policy
2. 深拷贝 policy 作为 ref (冻结, 不更新)
3. 准备 (chosen, rejected) 偏好对
4. 训练循环：
 - a. 计算 policy 和 ref 对 chosen/rejected 的 log-prob
 - b. 算 DPO loss
 - c. 反向传播, 只更新 policy

⚠ ref 模型在整个训练过程中保持不变——它是"锚点", 防止 policy 跑太远。

ORPO：无参考模型

ORPO (Odds Ratio Preference Optimization) 的核心创新：不需要参考模型！

核心思想

DPO 需要一个冻结的 ref 模型来计算 baseline。ORPO 用 odds ratio 替代：

```
`  
odds = P / (1-P)  
log_odds = log P - log(1-P)  
`
```

ORPO 的 loss：

```
`python  
def orpo_loss(policy_chosen_logps, policy_rejected_logps,  
chosen_n_tokens, rejected_n_tokens, orpo_lambda=1.0):
```

归一化：per-token 平均

```
chosen_mean = policy_chosen_logps / chosen_n_tokens.clamp(min=1)  
rejected_mean = policy_rejected_logps / rejected_n_tokens.clamp(min=1)
```

$\log \text{odds} = \log(p/(1-p))$

```
log_odds = (chosen_mean - _log1mexp(chosen_mean)) - \  
(rejected_mean - _log1mexp(rejected_mean))
```

$\text{OR loss} = -\log \text{sigmoid}(\log_odds)$

```
or_loss = -F.logsigmoid(log_odds).mean()
```

NLL loss — 在 chosen 上做 SFT

```
nll = -chosen_mean.mean()
```

总 loss = SFT + 偏好

```
loss = nll + orpo_lambda * or_loss  
return loss, chosen_mean, rejected_mean  
`
```

ORPO = SFT + 偏好对齐，一步完成：

- **nll**：在 chosen 上做 SFT（教模型生成好的回答）
- **or_loss**：用 odds ratio 拉大 chosen 和 rejected 的差距

⚠ **_log1mexp** 是数值稳定的 $\log(1-\exp(x))$ 实现，避免 $\log(0)$ 。

KTO：无成对数据

KTO (Kahneman-Tversky Optimization) 更进一步：不需要成对数据！

核心思想

基于 Kahneman & Tversky 的前景理论（Prospect Theory）：

- 人对"损失"比"收益"更敏感（损失厌恶）
- 只需要"好/坏"标签，不需要"哪个更好"的配对

```
python
def kto_loss(policy_chosen_logps, policy_rejected_logps,
ref_chosen_logps, ref_rejected_logps,
beta=0.1, desirable_weight=1.0, undesirable_weight=1.0):
chosen_logratio = policy_chosen_logps - ref_chosen_logps
rejected_logratio = policy_rejected_logps - ref_rejected_logps
```

KL baseline：所有 log-ratio 的均值

```
kl = torch.cat([chosen_logratio, rejected_logratio]).mean().clamp(min=0).detach()
```

非对称损失

```
chosen_losses = 1.0 - torch.sigmoid(beta * (chosen_logratio - kl))
rejected_losses = 1.0 - torch.sigmoid(beta * (kl - rejected_logratio))

loss = (desirable_weight * chosen_losses).mean() + \
(undesirable_weight * rejected_losses).mean()
return loss
```

前景理论的核心：

- chosen：让 log_ratio > kl（超过 baseline 才有正收益）
- rejected：让 log_ratio < kl（低于 baseline 才能惩罚）
- undesirable_weight 可以设更大（比如 1.5~2.0），体现"损失厌恶"

三种算法对比

维度	DPO	ORPO	KTO
参考模型	需要（冻结）	**不需要**	需要（冻结）
成对数据	需要	需要	**不需要**
训练复杂度	中	低	低
核心公式	log-prob ratio 差	odds ratio	前景理论
代表模型	Zephyr, Tulu-2	Llama-3, Qwen2	稀疏标注场景

怎么选？

- 有成对偏好数据 + 想要稳定 → DPO
- 有成对偏好数据 + 想省显存（不需要 ref 模型）→ ORPO
- 只有"好/坏"标签、没有配对 → KTO

隐式奖励：从 DPO 中提取

DPO 不需要显式的奖励模型，我们可以从训练好的 DPO 策略中"提取"隐式奖励：

,

$$r(x,y) = \beta * (\log \pi(y|x) - \log \pi_{\text{ref}}(y|x))$$

这个隐式奖励可以用来监控训练效果——chosen 的隐式奖励应该逐渐高于 rejected。

> 延伸对照 (LLMs-from-scratch) : rasbt ch07 的 [04_preference-tuning-with-dpo](#) 用 Llama 3.1 70B 生成偏好数据再从零

> 写 DPO——与我们"规则造偏好对"互补，想看真实偏好数据怎么来就读它。

课后练习

<details>

<summary>Q1: DPO 的 β 为什么不能太大？</summary>

A: β 控制"偏离参考模型的程度"。 β 太大意味着 KL 惩罚很重，策略被"锁死"在参考模型附近，学不到新东西。 β 太小则策略可能偏离太远，生成质量反而下降 ("reward hacking")。实际中 β 在 0.1~0.5 之间调优。

</details>

<details>

<summary>Q2: ORPO 为什么不需要参考模型？</summary>

A: DPO 需要 ref 模型来提供 baseline ("参考模型对 chosen/rejected 的偏好是什么")，然后优化"策略比参考模型更偏好 chosen"。ORPO 用 odds ratio 替代了这个 baseline——odds ratio 是 chosen 和 rejected 之间的直接比较，不需要外部参考点。同时 ORPO 的 NLL 项 (在 chosen 上做 SFT) 提供了"生成好回答"的信号，两者合起来完成了 SFT + 对齐。

</details>

<details>

<summary>Q3: Bradley-Terry 模型假设了什么？有什么局限？</summary>

A: Bradley-Terry 假设"偏好是传递的"——如果 $A > B$ 且 $B > C$ ，则 $A > C$ 。但人类偏好并不总是传递的 (比如你可能觉得 A 的风格比 B 好，B 的内容比 C 好，C 的风格比 A 好)。此外 Bradley-Terry 只建模了"二选一"的场景，无法直接处理"打分" (1~5 分) 或多选一。

</details>

课后作业

完成本章后，去 Assignment 8 完成题 4 (Bradley-Terry)、题 5 (DPO)、题 6 (ORPO)：

[Assignment 8](../../assignments/assignment_8/)

下一步

DPO 是"离线"算法——只用固定的偏好数据，不能从"尝试"中学习。下一步我们引入 PPO (在线 RL) 和 GRPO (不需要 Value Network 的 RL)，让模型能从自己的生成中不断改进。

[04 — 强化学习：PPO 与 GRPO](04_ppo_and_grpo.md)

04_ppo_and_grpo

04 — 强化学习：PPO 与 GRPO

> DPO 是"离线"的——只用固定数据训练。PPO 和 GRPO 是"在线"的——模型自己生成回答、获得奖励、从"尝试"中学习。PPO 是经典方案，GRPO 是 DeepSeek-R1 的选择。

前置知识

本章需要你已掌握：

- 03 章全部：Bradley-Terry 奖励模型、DPO 推导、forward_hidden()
 - 概率论基础：期望、方差、指数函数
- > 本章数学稍多，但每个公式都有直觉解释。不理解推导不影响读懂代码。

为什么 DPO 不够？

DPO 很好，但有一个根本限制：它是离线的。

	DPO（离线）	PPO/GRPO（在线）
数据来源	固定的偏好对	模型自己生成
学习方式	从"别人的回答"学	从"自己的回答"学
能否探索	不能	能
训练稳定性	高	较低（需要调参）
理论上限	受限于数据质量	可以超越数据

直觉：DPO 像是"看别人的考试答案学习"，PPO/GRPO 像是"自己做题、看对错、改进"。后者理论上更好，但更难训练。

PPO 核心思想

PPO（Proximal Policy Optimization）是最流行的 RL 算法之一。ChatGPT 的 RLHF 阶段就用了 PPO。

策略梯度的直觉

最简单的策略梯度公式：

、

$$\nabla J = E[A * \nabla \log \pi(a|s)]$$

、

- $\pi(a|s)$ ：策略在状态 s 下选择动作 a 的概率
- A ：advantage——这个动作比"平均"好多少
- $A > 0$ ：增大这个动作的概率
- $A < 0$ ：减小这个动作的概率

问题：更新太大容易崩

如果 advantage 很大，一步更新可能让策略剧变——从"经常选 A"变成"从不选 A"。这会导致训练不稳定。

解决：Clipped Surrogate

PPO 的核心创新——用 ratio clip 限制更新幅度：

```
python
def ppo_policy_loss(new_logp, old_logp, advantages, mask, clip=0.2):
    ratio = torch.exp(new_logp - old_logp) # 新旧策略比
    surr1 = ratio * advantages # 无截断
    surr2 = torch.clamp(ratio, 1-clip, 1+clip) * advantages # 截断后
    loss = -((torch.min(surr1, surr2) * mask).sum() / mask.sum())
    return loss
```

对应代码在 [06_ppo_training.py](../scripts/06_ppo_training.py)。

ratio clip 的直觉：

```
ratio =  $\pi_{\text{new}}(a|s) / \pi_{\text{old}}(a|s)$ 
```

- $\text{ratio} \approx 1$ ：策略变化很小，在"信任域"内，直接用 $\text{ratio} * A$
- $\text{ratio} >> 1$ ：策略变化太大，用 $(1 + \epsilon) * A$ 截断，防止剧变
- $\text{ratio} << 1$ ：策略变化太大（反方向），用 $(1 - \epsilon) * A$ 截断
- $\epsilon = 0.2$ ：论文推荐值，实际效果最好

loss

^

——+ / —————→ ratio

GAE：估计 Advantage

PPO 需要计算 advantage——"这个动作比平均好多少"。GAE（Generalized Advantage Estimation）是最常用的方法。

TD Error

```
 $\delta_t = r_t + \gamma * V(s_{t+1}) - V(s_t)$ 
```

- r_t ：t 时刻的奖励
- $V(s_t)$ ：状态 s_t 的价值估计（"从这个状态开始，未来能拿多少奖励"）
- γ ：折扣因子（控制"看多远"）

直觉： $\delta_t > 0$ 意味着"这个动作比预期好"， $\delta_t < 0$ 意味着"比预期差"。

GAE 公式

$$A_t = \sum_{l=0}^{T-t} (\gamma \lambda)^l \delta_{t+l}$$

从后往前递推：

```
python
def compute_gae(rewards, values, values_next, resp_mask, gamma=1.0, lam=0.95):
    B, L = rewards.shape
    adv = torch.zeros_like(rewards)
    lastgae = torch.zeros(B, device=rewards.device)
    m = resp_mask.float()

    for t in reversed(range(L)):
        nonterminal = m[:, t + 1] if t + 1 < L else torch.zeros(B, device=rewards.device)
        delta = rewards[:, t] + gamma values_next[:, t] - values[:, t]
        lastgae = delta + gamma lam nonterminal * lastgae
        adv[:, t] = lastgae

    returns = adv + values
    return adv, m, returns, m
```

$\gamma \lambda$ 权衡：

- $\gamma \lambda = 0$ ：只看单步 TD error $\delta_t \rightarrow$ 高 bias（估计不准）、低 variance（稳定）
- $\gamma \lambda = 1$ ：看完整轨迹 $\rightarrow$ 低 bias（更准）、高 variance（不稳定）
- $\gamma = 1.0, \lambda = 0.95$ ：实际中效果最好（LLM 通常 $\gamma = 1.0$ ，因为没有“终止状态”）

Value Head：估计 $V(s)$

GAE 需要 $V(s)$ ——状态价值函数。怎么得到？在 GPT backbone 上加一个“价值头”：

```
python
class TransformerWithValueHead(nn.Module):
    def __init__(self, transformer):
        super().__init__()
        self.transformer = transformer
        n_embed = transformer.lm_head.in_features
        self.value_head = nn.Sequential(
            nn.Linear(n_embed, n_embed),
            nn.ReLU(),
            nn.Linear(n_embed, 1),
        )
        nn.init.zeros_(self.value_head[-1].weight) # 零初始化
        nn.init.zeros_(self.value_head[-1].bias)

    def forward(self, idx):
        hidden = self.transformer.forward_hidden(idx)
        logits = self.transformer.lm_head(hidden) # Actor（策略）
        values = self.value_head(hidden).squeeze(-1) # Critic（价值）
        return logits, values
```

架构图：

```

idx (B, T)
v
GPT.forward_hidden() → hidden (B, T, n_embed)
v v
lm_head value_head
v v
logits values
(Actor) (Critic)

```

Actor-Critic 架构：Actor (lm_head) 决定"做什么", Critic (value_head) 评估"做得好不好"。两者共享 backbone, 但各自有独立的头。

PPO 训练循环

完整的 PPO 训练流程：

```

每一步:
1. Rollout: 用当前策略生成 response
2. Reward: 用奖励函数给 response 打分
3. KL Penalty: 减去 policy vs ref 的 KL 散度 (防止偏离太远)
4. GAE: 计算 advantage 和 returns
5. Multi-epoch update: 用 clipped surrogate 更新 N 个 epoch

```

对应代码的核心循环：

```

python
for step in range(ppo_steps):
1. Rollout: 生成 response

with torch.no_grad():
responses = [sft_model.generate(...) for _ in range(batch_size)]

```

2. 计算 old log probs

```

logits_old, values_old = ppo_model(sequences)
old_log_probs = per_token_log_probs(logits_old, sequences)

```

3. 奖励 + KL 惩罚

```

rewards = compute_reward(sequences, prompt_len)
kl = old_log_probs - ref_log_probs
rewards = rewards - kl_coeff * kl

```

4. GAE

```

advantages, returns = compute_gae(rewards, values_old, ...)

```

5. Multi-epoch update

```
for epoch in range(ppo_epochs):
    logits_new, values_new = ppo_model(sequences)
    p_loss = ppo_policy_loss(new_logp, old_logp, advantages, ...)
    v_loss = ppo_value_loss(values_new, values_old, returns, ...)
    entropy = compute_entropy(logits_new, ...)
    total_loss = p_loss - entropy_coeff entropy + 0.5 v_loss
    total_loss.backward()
```

△ PPO 每步都重新生成 response (on-policy)，然后在同一组数据上做 N 个 epoch 的更新。这比 DPO 复杂得多——需要维护 policy、ref、value 三个模型。

GRPO：不需要 Value Network

GRPO (Group Relative Policy Optimization) 是 DeepSeek-R1 的选择。核心创新：不需要 Value Network！

为什么可以去掉 Value Network？

PPO 需要 $V(s)$ 来计算 advantage。GRPO 用一个更简单的方法：

对同一个 prompt，采样 G 个回答，用组内平均奖励作基线。

```
python
def group_advantages(rewards, group_size, eps=1e-4):
    """A_i = (r_i - group_mean) / (group_std + eps)"""
    r = rewards.view(-1, group_size)
    mean = r.mean(dim=1, keepdim=True)
    std = r.std(dim=1, keepdim=True)
    adv = (r - mean) / (std + eps)
    return adv.reshape(-1)
```

对应代码在 [07_grpo_training.py](../scripts/07_grpo_training.py)。

Group Advantage 的直觉：

假设 prompt 是 $3+5=$ ，采样 $G=4$ 个回答：

```
回答 1: "8" → reward = 1.0 (正确)
回答 2: "7" → reward = 0.0 (错误)
回答 3: "9" → reward = 0.0 (错误)
回答 4: "8" → reward = 1.0 (正确)
```

```
group_mean = 0.5
group_std = 0.58
advantages: [+0.87, -0.87, -0.87, +0.87]
```

- 正确答案 $\text{advantage} > 0 \rightarrow$ 增大概率
- 错误答案 $\text{advantage} < 0 \rightarrow$ 减小概率

不需要 $V(s)$! 组内统计量 (mean、std) 就是天然的 baseline。

k3 KL 估计器

GRPO 用 Schulman 的 k3 估计器计算 per-token KL 散度：

```
python
def k3_kl(new_logp, ref_logp):
    """KL = exp(log_ref - log_new) - (log_ref - log_new) - 1"""
    diff = ref_logp - new_logp
    return torch.exp(diff) - diff - 1.0
```

k3 的三个优点：

- 无偏 (unbiased)：期望值等于真实的 KL 散度
- 非负：保证 $KL \geq 0$ (不需要 clamp)
- 数值稳定：不需要特殊处理

GRPO Loss

```
python
def grpo_loss(new_logp, old_logp, ref_logp, advantages, resp_mask, clip=0.2, kl_coef=0.04):
    adv = advantages[:, None] # broadcast over tokens
    ratio = torch.exp(new_logp - old_logp)
    surr1 = ratio * adv
    surr2 = torch.clamp(ratio, 1.0 - clip, 1.0 + clip) * adv
    surrogate = torch.min(surr1, surr2)
    kl = k3_kl(new_logp, ref_logp)
    per_token = surrogate - kl_coef * kl
    loss = -(per_token * resp_mask.float()).sum() / resp_mask.float().sum().clamp(min=1)
    return loss
```

和 PPO 的 clipped surrogate 几乎一样，只是 advantage 来自 group-relative 而非 GAE。

RLVR：可验证奖励

GRPO 最常配合 RLVR (RL with Verifiable Rewards) 使用：

```
python
def verify_answer(predicted, expected):
    """数学题答案正确 = 1, 错误 = 0"""
    numbers = re.findall(r'-?\d+\.?\d*', str(predicted))
    if not numbers:
        return False
    return abs(float(numbers[-1]) - float(expected)) < 0.01
```

RLVR 的核心思想：奖励来自可验证的规则，不需要训练 Reward Model。

奖励来源	例子	优点	缺点
人类标注	"这个回答好不好?"	灵活	贵、慢、有偏差
Reward Model	训练好的打分模型	自动化	需要训练、可能 reward hacking
RLVR	数学题答案对错	**免费、可靠**	只适用于可验证的任务

DeepSeek-R1 用 RLVR 训练推理能力——数学题答案对错、代码测试通过/失败，都是可验证的。

PPO vs GRPO 对比

维度	PPO	GRPO
Value Network	需要 (value_head)	**不需要**
额外参数	~50% (value_head)	0
显存	更多 (policy + value + ref)	更少 (policy + ref)
优势估计	GAE (γ, λ)	Group normalization
采样方式	每步生成一次	每个 prompt 生成 G 次
KL 惩罚	per-token KL penalty	k3 KL estimator
代表	InstructGPT, ChatGPT	DeepSeek-R1

怎么选？

- 有训练好的 Reward Model + 资源充足 → PPO
- 任务可验证（数学/代码）+ 想省资源 → GRPO
- 都不确定 → 先试 DPO（最简单）

> 延伸对照 (LLMs-from-scratch)：rasbt 续作
[reasoning-from-scratch](https://github.com/rasbt/reasoning-from-scratch)
> ch06 用裸 PyTorch 从零写 RLVR-GRPO (MATH-500 评测)，ch07 讲 DeepSeek-V3.2/Olmo3 风格
> 的进阶 GRPO 变体——Part 11 用工业框架 verl 跑同一批算法，一原理一工程双视角。

课后练习

<details>
<summary>Q1: GAE 的 λ 如何控制偏差-方差折中？</summary>
A: λ 控制"看多远的 TD error"。 $\lambda=0$ 只看当前步的 δ_t (高 bias: $V(s)$ 估计不准直接影响 advantage; 低 variance: 每步独立, 不累积误差)。 $\lambda=1$ 看完完整轨迹 (低 bias: 最终结果是最准的信号; 高 variance: 需要很多步才能得到 advantage, 每步的噪声累积)。 $\lambda=0.95$ 是经验最佳值——主要看近期的 TD error, 但也"稍微"考虑远期。
</details>

<details>
<summary>Q2: GRPO 为什么不需要 Value Network？</summary>
A: PPO 需要 $V(s)$ 来计算 "这个动作比预期好多少"。GRPO 换了一种思路——不问"比预期好多少", 而问"比同组其他回答好多少"。同一个 prompt 采样 G 个回答, 组内平均奖励就是天然的 baseline。这省掉了 Value Network (~50% 额外参数) 和 GAE 递推计算, 训练更简单、更省显存。
</details>

<details>

<summary>Q3: PPO 的 entropy bonus 有什么用？</summary>

A: $\text{entropy} = -\sum \pi(a|s) * \log \pi(a|s)$ 。熵越大 $\rightarrow$ 策略越"随机" $\rightarrow$ 探索越多。如果不加 entropy bonus, 策略可能很快收敛到"总是选同一个动作" (模式坍缩), 错过更好的回答。entropy_coeff=0.01 是个很小的系数——只需要"轻微鼓励"探索, 不要太多。

</details>

课后作业

完成本章后, 去 Assignment 8 完成题 7 (GAE) 和题 8 (PPO Clipped Loss) :

[Assignment 8](../assignments/assignment_8/)

下一步

训练完成后, 如何评估模型质量? 下一步我们用 GSM8K 数学题评估各阶段模型, 对比生成质量, 学习 temperature/top_k 等解码策略。

[05 — 评估与推理部署](05_eval_and_deploy.md)

05_eval_and_deploy

05 —

评估与推理部署：GSM8K、生成策略、完整流水线回顾

> 训练完成后, 如何量化模型质量? 本章用 GSM8K 数学题评估各阶段模型, 对比生成质量, 学习 temperature/top_k 等解码策略, 然后回顾完整的 LLM 后训练流水线。

学习目标

完成本章后, 你将能够：

- 搭建一条 GSM8K mini 评估流水线 (出题 $\rightarrow$ 生成 $\rightarrow$ 抽答案 $\rightarrow$ 对答案 $\rightarrow$ 算通过率)
- 解释 temperature/top_k/top_p 各自改变采样分布的哪一部分
- 对比 pretrain/SFT/DPO 三阶段模型在同一基准上的差异并归因

前置知识

• 必须掌握：01~04 章全部 (GPT-2 架构、预训练、SFT、奖励模型、DPO/PPO/GRPO——本章把各阶段模型放上同一评估台, 缺一段就看不出差异从哪来)

• 建议掌握：Part 6 的生成 (自回归生成、temperature 采样——解码策略对比节直接建立在其上)

- 可选：NumPy 手写 softmax (无也不影响, 代码用 torch)

> 本章是 Part 8 的收尾——把前面所有阶段串起来, 看"从预训练到部署"的全链路。

GSM8K 评估

GSM8K (Grade School Math 8K) 是评估数学推理能力的标准 benchmark。我们用类似的方法评估模型：生成答案 → 提取数字 → 对比金标 → 计算准确率。
对应代码在 [08_eval_and_chat.py](../scripts/08_eval_and_chat.py)。

评估流程

```
`python
def evaluate_model(model, stoi, itos, problems, max_tokens=10, temperature=0.8, top_k=10):
    correct = 0
    for prompt_text, expected in problems:
        prompt_ids = [stoi.get(c, 0) for c in prompt_text]
        prompt_tensor = torch.tensor([prompt_ids], device=device)

        with torch.no_grad():
            gen = model.generate(prompt_tensor, max_new_tokens=max_tokens,
                                temperature=temperature, top_k=top_k)

        resp_ids = gen[0].tolist()[len(prompt_ids):]
        resp_text = ''.join(itos.get(tid, '?') for tid in resp_ids)
        predicted = extract_number(resp_text)

        is_correct = (predicted is not None) and abs(predicted - expected) < 0.01
        if is_correct:
            correct += 1

    return correct / len(problems)
`
```

三步走：

1. 用 prompt 编码问题（如 3+5=）
2. 让模型生成回答
3. 从回答中提取数字，与期望答案比较

△ 我们的模型很小（CPU 模式 ~0.1M 参数），数学能力有限。评估的目的是看趋势（各阶段的相对改进），而不是追求绝对准确率。

全阶段对比

理论上，各阶段模型的数学能力应该是：

```
`
Base (随机) < Pretrain (续写) < SFT (对话) < DPO/PPO/GRPO (对齐)
~0% ~5-10% ~10-20% ~15-30%
`
```

实际效果取决于数据量、模型大小、训练步数。我们的 CPU 缩小版数字会更低，但趋势应该一致。

生成参数：控制"创造性 vs 确定性"

生成时有几个关键参数控制输出的"风格"：

Temperature

```
`python
logits = logits[:, -1, :] / temperature
probs = F.softmax(logits, dim=-1)
```

temperature	效果	适用场景
0.1~0.3	非常确定，几乎总是选最高概率的 token	数学、代码
0.7~0.9	平衡多样性和连贯性	日常对话
1.0	原始分布，不做缩放	默认值
1.5+	非常随机，可能不通顺	创意写作

直觉：temperature 除以 logits，相当于"拉平"或"拉尖"概率分布。低温度让高概率 token 更突出（确定性），高温度让低概率 token 也有机会（多样性）。

Top-K

```
`python
if top_k is not None:
    v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
    logits[logits < v[:, [-1]]] = -float('Inf')
```

只保留概率最高的 K 个 token，其余设为 **-inf**（softmax 后变成 0）。

top_k	效果
1	贪心解码：每次选最可能的 token
5~10	较保守：候选少，质量稳定
40~50	较多样：候选多，更有趣

Top-P（Nucleus Sampling）

Top-K 的问题是"固定候选数"——不管概率分布是集中还是分散。Top-P 改为"累积概率阈值"：

- 1. 将 token 按概率从高到低排序
- 2. 从最高概率开始累加，直到累积概率 >= p
- 3. 只在这些 token 中采样

Top-P	效果
0.9	保留累积概率 90% 的 token
0.95	更多样
1.0	退化为原始采样

Top-K vs Top-P：Top-K 固定候选数，Top-P 动态调整。概率集中时 Top-P 选得少，分散时选得多——更自适应。

⚠ 本教程的 `generate()` 只实现了 temperature + top_k，未实现 top_p（留作练习）。

Repetition Penalty

生成时模型容易陷入重复（"the the the..."）。Repetition penalty 通过惩罚已出现过的 token 来缓解：

```
python
伪代码（本教程未实现）

for token_id in generated_tokens:
    logits[token_id] /= repetition_penalty # 降低已出现 token 的概率
```

推荐的推理超参数

场景	temperature	top_k	说明
数学/代码	0.0~0.3	1~5	确定性高，减少错误
日常对话	0.7~0.9	10~50	平衡多样性和连贯性
创意写作	1.0~1.5	50+	高多样性，更多创意
翻译/摘要	0.3~0.5	10~20	准确为主，少随机性

KV Cache：推理加速

自回归生成时，每生成一个 token 都要重新计算整个序列的注意力——但前面 token 的 Key/Value 不变。KV Cache 缓存它们，避免重复计算。

```

没有 KV Cache（每步都重算全部）：
step 1: [A] → K_A, V_A
step 2: [A, B] → K_A, V_A, K_B, V_B ← K_A, V_A 重算了
step 3: [A, B, C] → K_A, V_A, K_B, V_B, K_C, V_C ← 全部重算

有 KV Cache（只算新 token）：
step 1: [A] → cache: K_A, V_A
step 2: [B] → 只算 K_B, V_B，拼接到 cache
step 3: [C] → 只算 K_C, V_C，拼接到 cache
```

本教程的 `generate()` 没有实现 KV Cache（教学用，代码更清晰）。生产环境的推理引擎（vLLM、TGI）都有 KV Cache。

完整流水线回顾

从零到部署的完整 LLM 后训练流程：

预训练（续写能力） 数据：大规模无标注文本 损失：next-token prediction cross-entropy 脚本：02_pretrain.py
SFT（对话能力） 数据：(instruction, response) 对 损失：response 部分的 cross-entropy（Prompt Masking） 脚本：03_sft.py
对齐（人类偏好） 路径 A：DPO/ORPO/KTO（离线，简单） 数据：(chosen, rejected) 偏好对 脚本：05_dpo_alignment.py 路径 B：PPO（在线，需要 Reward Model） 数据：prompt + reward model 打分 脚本：06_ppo_training.py 路径 C：GRPO（在线，RLVR 可验证奖励） 数据：prompt + verifier 判定对错 脚本：07_grpo_training.py
评估（量化效果） 方法：GSM8K 准确率、生成质量对比、交互式测试 脚本：08_eval_and_chat.py

三条对齐路径

路径	复杂度	数据需求	适用场景	代表
DPO/ORPO/KTO	低	成对偏好	通用对齐	Zephyr, Llama-3
PPO	高	Reward Model	任意奖励	InstructGPT, ChatGPT
GRPO	中	可验证规则	数学/代码	DeepSeek-R1

脚本运行顺序

推荐的训练流程：

```
bash
```

1. 预训练

python 02_pretrain.py → ckpt_pretrain.pt

2. SFT

```
python 03_sft.py → ckpt_sft.pt
```

3. 对齐（三选一）

```
python 05_dpo_alignment.py → ckpt_dpo.pt # 最简单
```

或

```
python 06_ppo_training.py → ckpt_ppo.pt # 更强大
```

或

```
python 07_grpo_training.py → ckpt_grpo.pt # DeepSeek 风格
```

4. 评估

```
python 08_eval_and_chat.py → 对比所有阶段
```

下一步：Scaling Up

本教程用 CPU 缩小版演示了完整的后训练流程。如果你想进一步：

方向	怎么做
更大模型	增大 n_embed、n_head、n_blocks（需要 GPU）
更多数据	用真实对话数据集（ShareGPT、UltraChat）
多 GPU	用 FSDP 或 DeepSpeed ZeRO 分布式训练
更长上下文	增大 context_length，用 RoPE 替代 learned PE
更好的 tokenizer	用 BPE（tiktoken 或 HuggingFace tokenizers）

核心理想不变——只是规模更大、组件更现代。

课后练习

```
<details>
<summary>Q1: Temperature=0 和 top_k=1 的效果一样吗？</summary>
A: 不完全一样。temperature=0 让 logits 除以 0（变成 inf），softmax 后全部概率集中在最大值上——等价于 greedy。top_k=1 也是只保留最高概率的 token。两者效果在大多数情况下相同，但 temperature=0 在多个 token 概率相同时行为取决于实现（可能选第一个），top_k=1 在 tie-breaking 时也有类似问题。实际中通常用 temperature=0.01 而不是严格的 0，避免数值问题。
</details>

<details>
<summary>Q2: 为什么 GSM8K 评估要固定 random seed？</summary>
A: 因为生成涉及随机采样（temperature > 0 时），同样的模型在不同 seed 下可能给出不同答案。固定 seed 保证结果可复现——你跑两次得到的准确率一样。这也是为什么评估时通常用较低的 temperature（减少随机性）或多次采样取平均。
```

</details>

<details>

<summary>Q3: DPO、PPO、GRPO 三条路径怎么选？</summary>

A: 取决于你的场景。如果你有成对偏好数据且想要简单稳定，选 DPO。如果你有训练好的 Reward Model 且资源充足，选 PPO（理论上上限更高）。如果你的任务是可验证的（数学、代码），选 GRPO（最简单、最省资源）。实际中很多人先试 DPO，不够再上 PPO/GRPO。

</details>

恭喜你完成了 Part 8 的全部学习！你已经掌握了 LLM 后训练的完整流程——从预训练到 SFT、从奖励模型到 DPO/PPO/GRPO、从评估到部署。

这套流程是 ChatGPT、Llama、DeepSeek-R1 等模型背后的核心技术栈。虽然我们用的是 CPU 缩小版，但原理和工业级训练完全一样。

[← 上一章：强化学习 PPO 与 GRPO](04_ppo_and_grpo.md) | [下一章：推理与服务] (06_inference_and_serving.md) | [Part 8 README](README.md)

06_inference_and_serving

06 — 推理与服务：从"能聊"到"能上线"

- > 05 章的 `08_eval_and_chat.py` 让模型能对话了，但"能聊"和"能上线"之间隔着一整门
- > 工程学科：**怎么让它更小（量化）、更快（投机解码/批处理）、更省（KV 显存管理），
- > 以及怎么用数字向别人证明它快（TTFT/TPOT）**。本章全部概念都有配套实测：
- > 跑一遍 `[scripts/09_quantize_and_serve.py](../scripts/09_quantize_and_serve.py)`（CPU 可跑，2-3 分钟）。

学习目标

完成本章后，你将能够：

- 计算 KV Cache 显存公式并说明 GQA/量化各省多少
- 解释 量化（RTN int8/int4）、投机解码、连续批处理各自解决什么瓶颈
- 测量 TTFT/TPOT/吞吐三指标并用它们对比两个推理引擎

前置知识

- 必须掌握：Part 7 03 章（KV Cache、GQA——本章算 KV 显存时直接用）
- 建议掌握：Part 9 02 章（memory-bound vs compute-bound——本章所有优化技巧的共同原理）；05 章生成参数（temperature/top_k——投机解码建立在它之上）
- 可选：GPU 显存层次（SMEM/HBM）——量化与批处理的分析会更立体

0. 一个原理打全部：decode 是 memory-bound

自回归生成的每一步都要把全部权重从显存读一遍，算术强度 $\approx 1 \text{ FLOP/字节}$ ——远低于 GPU 的平衡点（4090 约 1:150）。所以 decode 阶段 GPU 的计算单元大部分时间在

等权重到位。整个推理优化业界的招数都能归结为一句话：

- > 让每一次权重搬运多干点活：量化（搬运的字节变少）、批处理（一次搬运服务多个请求）、
- > 投机解码（一次搬运验证多个 token）、KV 管理（别让没用的 KV 挤占搬运带宽）。

1. 权重量化：int8 / int4

RTN（round-to-nearest）对称量化，本课脚本的实现：

```
`  
scale = max|W| / 127 # 逐输出通道（一行一个 scale）  
W_int8 = round(W / scale) # 反量化 W' = W_int8 × scale，误差 ≤ scale/2  
`
```

配置	有效 bits/权重	典型 ppl 代价（7B 级）
fp16 基线	16	—
int8 逐通道	~8.1	**几乎无损**（LLM.int8()：<0.05）
int4 分组 g128	~4.1	+0.1~0.3（GPTQ） / +0.2~0.4（AWQ）
int4 无分组	4	大模型上会崩（离群通道）

- 分组的意义：group size=128 时每 128 个权重配一个 fp16 scale（摊 0.125 bit），离群的通道自己一组，不污染邻居。大模型存在少数"离群激活通道"（LLM.int8 的发现），逐张量一个 scale 会被离群值拖垮整体分辨率。
- GPTQ（Frantar 2022）：逐层用 Hessian 误差补偿——量化一列后把误差摊给未量化的列，不用反传、一遍校准数据搞定。AWQ（Lin 2023）：发现"重要通道"跟着激活幅值走，量化前按激活统计给通道做等价缩放（ $s = \text{mean}|x_{\text{act}}|^{\alpha}$ ），无反传可扩到 175B。
- 一句话对比：GPTQ 事后补偿误差，AWQ 事前保护重要通道。
- ⚠ 本课 ~0.4M 玩具模型的实测（脚本①，GPU 500 步训练）：int8 $\Delta \approx +0.4$ 、int4 g128 $\Delta \approx +0.3$ （7B 论文里 int8 <0.05）——模型越小、训练越不充分，对量化越敏感；而且小模型没有离群通道，int4 分不分组差别不大——离群值是规模涌现的。把训练步数翻倍再看， Δ 会变小：量化损伤与训练充分度负相关，这本身就是个可写进面经的观察。
- ⚠ CPU 路径（脚本默认只训 50 步）模型严重欠训练， Δ 会变小甚至变负——对比实验请在 GPU 路径跑。

真实模型量化环境自检（bitsandbytes 是 CUDA 专用库，装错版本能卡一下午）：

```
`  
python  
import torch, bitsandbytes as bnb # 版本要匹配 torch 的 CUDA（cu121↔cu121）  
from transformers import AutoModelForCausalLM  
m = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-0.5B-Instruct",  
load_in_4bit=True, device_map="auto") # 一行 4bit；Colab 免费 T4 可跑  
`
```

2. KV Cache 显存：GQA 和量化各省多少

```
`  
  
KV_bytes = 2(K+V) × n_layers × n_kv_heads × head_dim × seq_len × batch × bytes  
`
```

配置 (LLaMA-7B 级, seq 2048, bs=1, fp16)	KV 显存
MHA (kv_heads=32)	1.07 GB
GQA (kv_heads=8)	**0.27 GB** ← Part 7 GQA 的直接意义
GQA + KV int8	0.13 GB
GQA + KV 2bit (KIVI)	~0.03 GB

- KIVI (ICML 2024) 的细节：K 按通道 (feature 维) 量化、V 按 token 量化——因为 K 存在少数固定的离群通道 (所有 token 一致)，必须按通道隔离；V 没有该现象。另外保留开头几个 + 最近 token 的全精度窗口 (attention-sink)。
- 跑 [脚本②](../scripts/09_quantize_and_serve.py) 的计算器，把你自己的数字算出来。

3. PagedAttention 与连续批处理 (vLLM 的两大支柱)

问题：给每个请求按 `max_seq_len` 预留一整块连续 KV，实际生成长度参差 → vLLM 论文实测 60-80% 的 KV

显存被浪费 (内部碎片：预留没用完；外部碎片：零散小空隙插不进新请求)。

PagedAttention：学操作系统虚拟内存——KV 切成 16 token/块，逻辑块表映射物理块，按需分配、写时复制；相同前缀的请求共享物理块 (prefix sharing)。浪费降到 <4%。

连续批处理 (Orca, OSDI'22)：静态 batch 要等最长的请求跑完才能换人 (早完成的请求占着 GPU 空转)；iteration-level 调度每个 decode 步都允许新请求进/完成请求出，batch 常满。Orca 报告同延迟下吞吐 $36.9\times$ (对比 FasterTransformer)。

[脚本③](../scripts/09_quantize_and_serve.py) 用简化模拟量化了这个叙事 (64 请求)：整块预留浪费 41% → 分页 5%——方向与论文一致 (60-80% 来自真实长尾负载 + 外部碎片)。

4. 投机解码：用小模型给大模型"代笔"

机制 (Leviathan et al. 2023)：

循环：

1. draft (小模型) 自回归采样 γ 个 token (顺带记下每个位置的分布 p_d)
 2. target (大模型) 一次前向，并行给出 γ 个位置的分布 p_t
 3. 逐个验证： $u \sim U(0,1)$, $u < p_t/p_d \rightarrow$ 接受；否则从 $\max(0, p_t - p_d)$ 重采样并终止
- 证明：最终输出分布与 target 单独采样完全一致 (无损！)

为什么快？memory-bound：验证 γ 个 token 的一次前向 $\approx$ 生成 1 个 token 的钱 (权重只读一遍)。期望每周期产出 $E = (1 - \alpha^{(\gamma+1)}) / (1 - \alpha)$, α 是接受率。

[脚本④](../scripts/09_quantize_and_serve.py) 的实测 (draft=减半宽单层, $\gamma=4$)：
 $\alpha \approx 0.60 \rightarrow$ 实测 ≈ 2.47 tokens/cycle (121 token/49 次前向)，理论公式给 2.31——
公式与实测方向一致 (差异来自每周期"白赚 token"的分布波动)。
⚠ draft 太弱 (α 低) 会不赚反赔：draft 的 γ 次前向白花。

5. 部署指标与 vLLM 最小实操

指标	定义	谁在乎
TTFT	首 token 延迟 (prefill 主导)	用户体验"反应快不快"
TPOT/TBT	平均每 token 间隔 (decode 主导)	打字机流式体验
吞吐	全部请求 tokens/s 合计	成本
goodput	满足 SLO 的吞吐 (如 P99 TTFT $\leq$ 200ms 且 TPOT $\leq$ 50ms)	生产答辩用

E2E 延迟 $\approx$ TTFT + TPOT $\times$ (输出 token 数 - 1)。批越大吞吐越高、但 TTFT/TPOT 变差——这就是 goodput 存在的原因。

10 行上手 vLLM (模拟里的概念对号入座)：

```
`bash
pip install vllm # 需 Linux + NVIDIA GPU ; Colab T4 可跑 0.5B 模型
vllm serve Qwen/Qwen2.5-0.5B-Instruct --max-model-len 2048 # 起一个 OpenAI 兼容服务
curl http://localhost:8000/v1/chat/completions -H 'Content-Type: application/json' \
-d '{"model": "Qwen/Qwen2.5-0.5B-Instruct", "messages": [{"role": "user", "content": "你好"}]}'`
```

概念映射：`--max-model-len` $\rightarrow$ KV 池大小；PagedAttention $\rightarrow$ 自动开启 (看日志 `# GPU blocks`) ；
连续批处理 $\rightarrow$ 服务端自动；`--gpu-memory-utilization` $\rightarrow$ KV 池占比调参。
对比实验：同一批 100 个请求分别用 `transformers` 生成循环和 vLLM 打服务，测吞吐差 (通常 5-20 $\times$)。

学完本章你能...

- 用 memory-bound 一句话解释量化/批处理/投机解码为什么有效
- 手算 KV 显存，说出 GQA 和 KV 量化各省多少
- 画出 PagedAttention 的块表，解释 60-80% $\rightarrow$ <4% 的来源
- 写出投机解码的接受判据和期望产出公式，并复述实测对照
- 说出 TTFT/TPOT/goodput 的定义与取舍，并起一个 vLLM 服务

课后练习

```
<details>
<summary>Q1: 为什么量化 int8 基本无损而 KV cache 量化更难？</summary>
A: 权重是静态的、可以离线校准分组，逐通道 scale 吸收离群；KV 是运行时逐 token 增长的，每步都要量化-反量化（有额外 kernel 开销），且 K 的离群是"跨 token 一致的固定通道"，必须按通道量化（KIVI），实现更绕。另外注意力对 KV 误差更敏感（softmax 后误差被放大）。
</details>

<details>
<summary>Q2: 一个 SLO 要求 P99 TTFT  $\leq$  200ms。批开得越大越接近还是越远？怎么办？</summary>
A: 越远——批大  $\rightarrow$  排队 + prefill 变长  $\rightarrow$  TTFT 恶化。解法：admission control（限流进 batch）、chunked prefill（把长 prompt 的 prefill 切片，decode 夹在里面）、按 SLO 分池。
这就是 goodput 指标存在的原因：吞吐要在"满足 SLO 的请求"上算。
</details>

<details>
<summary>Q3: 投机解码在 batch 很大时还赚吗？</summary>
A: 不赚。大 batch 时系统转为 compute-bound（算力饱和），target 一次验证  $\gamma$  个 token 不再"顺带免费"，draft 反而抢算力。投机解码是 decode、小 batch、内存墙场景的武器。
</details>
```

课后作业

[Assignment 8](../assignments/assignment_8/) (综合题不变) + 跑通
[scripts/09_quantize_and_serve.py](../scripts/09_quantize_and_serve.py) 的四个实验并记录你的实测数字。

下一步

模型上线前还有最后一道关：怎么科学地知道它变好/变坏了？

下一章讲评估学——benchmark 污染、LLM-as-judge 的偏差、以及为什么 ppl 不能跨模型比较。

[07 — 评估学：怎么科学地给模型打分](07_evaluation.md)

07_evaluation

07 — 评估学：怎么科学地给模型打分

- > "我们的模型 GSM8K 提升了 3 个点"——这句话可能是真的进步，也可能是背过题库。
- > 05 章我们跑过 GSM8K mini 评估，但评估的方法论（怎么测、信多少、怎么防作弊）是独立的
- > 一门学问。这是对齐/后训练岗面试的常客，也是看论文时必备的"免疫力"。
- > 本章还有下半场：幻觉与安全——怎么证明模型在瞎编（语义熵）、对齐的校准代价（ECE）、
- > 拒绝行为的机理（refusal direction）、以及国内上线的合规四件套（配套脚本 11/12）。

学习目标

完成本章后，你将能够：

- 解释 三种评估范式（规则/人工/LLM-judge）各自的适用场景与坑
- 操作 lm-evaluation-harness：跑标准基准、控制 fewshot/seed 四元组、写自定义 task
- 手写 幻觉检测三件套——语义熵（SE）、温度 sweep、ECE 校准度量（脚本 11）
- 描述 拒绝行为的"单方向假说"与 diff-in-means/方向消融方法（不跑敏感实验）
- 列出 国内生成式 AI 上线的合规四件套与各自依据

前置知识

- 必须掌握：05 章（08_eval_and_chat.py 的 GSM8K 流程——出题→生成→抽答案→对答案，本章的评估讨论都从它出发）；03/04 章（DPO/PPO/GRPO 概念——§ 6.4 校准对比和 § 7 安全对齐直接建立在其上）
- 建议掌握：06 章（部署视角——评估是上线前的最后一道闸，§ 8 合规与之衔接）
- 可选：Qwen2.5-0.5B (-Instruct) 已缓存（脚本 11/12 用它做真实模型实验，缺失时脚本会打印下载指引并优雅退出）

1. 三种评估范式，各有各的坑

范式	代表	优点	坑
规则评估	exact match、选择题 loglikelihood、ppl	便宜、可复现、客观	只能测"有标准答案"的能力；容易被背题
人工评估	Chatbot Arena（成对盲测→Elo）	最接近真实偏好，难作弊	慢、贵、噪声大
LLM-as-judge	MT-Bench、AlpacaEval	便宜、可规模化	评审模型有**位置偏差、长度偏差、自偏好**

- LLM-judge 与人类的一致率其实不低：GPT-4 judge 与人类 ~85%（去平票后），甚至高于人类之间的一致率 81%。但必须做位置交换（A/B 轮流放前后）和长度控制（AlpacaEval 2.0 的 length-controlled win-rate）——否则"哪个答案长哪个赢"。
- ppl 的陷阱：perplexity 依赖 tokenizer，不同词表的模型之间不可比（6400 词表的 ppl 11 和 15 万词表的 ppl 3 没有可比性）。跨模型请看同一语料的 bits-per-byte（bpc）。

2. lm-evaluation-harness：预训练评估的事实标准

EleutherAI 的 [lm-eval-harness](https://github.com/EleutherAI/lm-evaluation-harness) 统一了 300+ 任务，只做两类请求：

- loglikelihood：选择题——算每个选项的归一化对数概率，选最高的（注意归一化方式：
`acc` vs `acc_norm` 按字符长度归一，结果可能差好几个点）
- generate_until：生成题——生成后抽取/匹配答案（如 GSM8K 抽最后一个数字）
（另有 ppl 类任务用的 `loglikelihood_rolling` 变体）

minimind 官方就是用它报告的（ceval 24.89 / cmmlu 25.38 / arc_easy 28.49 / piqa 50.65）。
复现 minimind 后跑同一套任务，你的模型分数落在 25-30%（多选题≈随机 25%）是正常的——
2.9GB 语料训出的 26M 模型本来就不是为刷榜而生的，看的是相对变化和 pipeline 是否可信。

实操：把 lm-eval 当库用——arc_easy 基线 + 自定义 task（脚本 12）

- > 配套脚本：`[scripts/12_lm_eval_hands_on.py](../scripts/12_lm_eval_hands_on.py)` +
`[mytasks/](../scripts/mytasks/)`（自定义 task：yaml 配置 + jsonl 数据）。
- > 实测环境：lm_eval 0.4.13, Qwen2.5-0.5B-Instruct fp16, RTX 4090, limit=100 每档不到 1 分钟。

先避雷（都是真实踩过的坑）：

- ⚠ 安装：v0.4.10 起 `pip install lm_eval` 默认不装 HF 栈（transformers/datasets），
`import lm_eval` 能过但 `model="hf"` 运行时才报错——必须 `pip install "lm_eval[hf]"`
（本课实测 0.4.13, requirements.txt 的可选依赖区已注明）。
- ⚠ Instruct 模型的 chat template 警告：跑 arc_easy 时 harness 会提示
`appears to be an instruct or chat variant but chat template is not applied`。
对 loglikelihood 选择题这是预期行为（与公开的 base 模型分数同口径可比）；
要评对话能力再开 `apply_chat_template`（那会换基线，见下面的互比坑）。
- ⚠ `trust_remote_code` 是 `--model_args` 的成员，不是 CLI 顶层参数：
`--model_args pretrained=xxx,trust_remote_code=True`（自定义架构必踩）。
- ⚠ `--batch_size auto` 可能 OOM：自动探测按最大 batch 试探。用 `auto:N`
（如 `auto:8`）= 带 N 倍上限的探测，大模型/小显存首选。
- ⚠ 随机性是四元组：`'random_seed=0, numpy_random_seed=1234, torch_random_seed=1234, fewshot_random_seed=1234'`（simple_evaluate 默认值）。fewshot seed 决定抽哪几道示例题——
换它 = 换题 = 分数会动；复现实验要四个全钉住并写进实验记录。

Python API 三步走（脚本 12 的核心，比 CLI 好嵌入训练管线）：

```
`python
```

```
import lm_eval
result = lm_eval.simple_evaluate(
    model="hf",
    model_args="pretrained=Qwen/Qwen2.5-0.5B-Instruct,dtype=float16,batch_size=16",
    tasks=["arc_easy"], num_fewshot=0, limit=100,
    verbosity="ERROR", log_samples=False, bootstrap_iters=0,
)
print(result["results"]["arc_easy"]["acc,none"]) # acc 指标默认聚合键
```

实测输出（0-shot vs 5-shot 同任务对比）：

```
arc_easy (limit=100, 0-shot): 58.0%
arc_easy (limit=100, 5-shot): 59.0%
```

- > fewshot 数变了，基线就变了：0-shot 的 58.0% 和 5-shot 的 59.0% 是两条基线，
- > 不能说“加了 5-shot 提升 1 个点”。要比较模型/训练阶段，必须固定同一 num_fewshot
- > （以及同一 limit、同一种子四元组）。小样本下 1 个点差距 < 噪声，别过度解读。

自定义 task（把课程知识变成 benchmark，mytasks/course_quiz）：

```
yaml
task: course_quiz
dataset_path: json
dataset_kwargs: {data_files: {test: course_quiz.jsonl}}
test_split: test # ⚠ 坑：不写这行，0.4.13 直接报 "must have valid or test docs"
output_type: multiple_choice
doc_to_text: "Q: {{question}}\nA:"
doc_to_target: "{{answer}}" # jinja 渲染成 "2" 会被转回 int 作为 gold 下标
doc_to_choice: "{{choices}}"
metric_list: [{metric: acc}]

python
from lm_eval.tasks import TaskManager
tm = TaskManager(include_path="../../../mytasks") # 绝对路径，别依赖 cwd
result = lm_eval.simple_evaluate(model="hf", model_args=...,
tasks=["course_quiz"], task_manager=tm)
```

两个实测坑（都写进了脚本注释）：

1. ⚠ data_files 相对路径以“运行时的 cwd”为基准解析，不是 yaml 所在目录——从项目根跑 `--include_path mytasks/` 会找不到 jsonl。脚本 12 的做法：**运行时在系统临时目录生成一份绝对路径版 yaml**（data_files 指向 jsonl 的绝对路径）再交给 TaskManager，从任何目录跑都成立；仓库里的 `mytasks/course_quiz.yaml` 保持相对路径规范形态（在 `mytasks/` 目录下可直接给 `lm_eval --include_path .` 用），不被脚本改动。
2. ⚠ test_split: test 必须显式声明——data_files 里写了 test：也不会自动推断。

实测输出：`course_quiz (自定义 5 题): 0.0%`——0.5B-Instruct 对 DPO/GRPO/LoRA/量化/RAG 概念题五题全错。这不是管线 bug：手动复算逐选项 loglikelihood，模型确实偏好错误选项（未归一化的 loglikelihood 偏爱“读起来顺”的通用短说法，正确答案往往是更具体的技术表述）。这个 0 分恰好演示了自定义 task 的意义：你的课程/业务知识，模型没学过就是没学过——同一模型 arc_easy 58%、自家题库 0%，评估永远只测“训练分布里有的东西”。

（也给"写选项时别让正确答案天然更长"这条经验做了注脚：先排除长度混淆，再谈对错。）

vLLM 后端（吞吐显著高于 hf，连续批处理；⚠ 本课环境未装 vLLM，命令为预期行为，供有 vLLM 环境时参考）：

```
`bash
lm_eval --model vllm \
--model_args pretrained=Qwen/Qwen2.5-0.5B-Instruct,dtype=auto,gpu_memory_utilization=0.8 \
--tasks arc_easy --limit 100 --batch_size auto
`
```

> ⚠ v0.4.12 起 vLLM 最低版本要求变严，版本不匹配会直接拒绝加载——升级 harness
> 时记得连 vLLM 一起看版本矩阵。

替代品格局（工具选型一句话）：

工具	生态位	什么时候选它
lm-eval-harness	英文学术事实标准（300+ 任务，论文标配）	复现论文数字、投英文会议
OpenCompass	中文生态最全（CEval/CMMLU 等基准 + 司南榜单）	中文模型、国内汇报口径
lighteval	HuggingFace 出品，与 transformers/datasets 深度集成	HF 技术栈生产线内嵌评测

方法论总参考：Biderman et al., Lessons from the Trenches on Integrating LLMs
（arXiv 2405.14782，lm-eval-harness v0.4 论文）——为什么"统一请求协议 + 社区任务库"
是评估可信度的基石。

3. HELM：一次看七个维度

Stanford [HELM](https://arxiv.org/abs/2211.09110) 的贡献是把"单一排行榜分数"拆成
指标 × 场景矩阵：accuracy 之外还有 calibration（模型说 70% 把握的事是否真有 70% 对）、
robustness、fairness、bias、toxicity、efficiency。面试里能说出"除准确率外我会看校准和
鲁棒性"就已经超出多数候选人。

4. Benchmark 污染：不是理论问题，是实测问题

污染 = 测试题（或其近似）混进了训练数据 → 分数衡量的是记忆力而非能力。

- 硬证据 GSM1k（arXiv 2405.00332）：研究者造了一套"GSM8K 的镜像新题"（同难度同格式，
确保没泄露），一批开源模型分数应声下跌：Mistral - 8%，Phi - 21%——说明部分"数学能力"
是背出来的。
- 常规防御：训练数据 vs 测试集做 n-gram 重叠去污染（GPT-3 用 13-gram，Llama 用 20-gram）；
进阶检测：embedding 检索相似题、Min-K% Prob（成员推断）、直接让模型复述测试题。
- 我们 Part 8 的做法诚实版：GSM8K 测试集只用于评估、训练用的是算术合成数据——
但要小心：如果合成数据的题型和 GSM8K 高度同构，也是一种"软污染"。

5. 给自己的模型搭一条"最小可信评估线"

不需要大而全，四条就够（都是本课程已有的能力）：

,

1. ppl/bpc：held-out 文本，固定 tokenizer（Part 7 的 09_eval_demo.py）

- 2. 任务集：GSM8K/CEval 各抽 100 题固定种子（Part 8 的 08 脚本模式）
- 3. 对照组：每阶段 ckpt 都测（Base/SFT/DPO 分开报数——单点分数无意义）
- 4. 防污染声明：训练数据与评测集的重叠检查一句话写进实验记录

> 面试叙事模板：“我评估模型时固定种子与题集、分阶段对照、并做过 n-gram 重叠检查”——> 这句话的含金量高于“我的模型 GSM8K 到了 X 分”。

6. 幻觉：把"感觉它在瞎编"变成数字

- > 配套脚本：`[scripts/11_hallucination_safety.py](../scripts/11_hallucination_safety.py)`（实验 A/B/C，GPU 约 5 分钟）。
- > 本节所有数字来自真实运行：RTX 4090（24GB）单卡，Qwen2.5-0.5B-Instruct fp16，
- > torch 2.6.0+cu124 / transformers 4.57.6，seed=1337，采样 n=10、T=0.7（2026-09-01 实测）。

6.1 机理：下一 token 预测的结构性根源

预训练目标是似然最大化，不是真值对齐：模型学的是 $p(\text{下一个 token} \mid \text{上文})$ ，“贝加尔湖最深处是1642米”和“贝加尔湖最深处是6385米”在训练语料里都是“流畅的中文”。推理时模型对任何问题都会给出概率最高的续写——它没有被训练出“我不知道”的默认行为，于是知识边界外的流畅胡编（confabulation）是结构性的，不是 bug。

幻觉的三个来源（面试分层答）：① 知识缺失（参数里没存）→ 编；② 知识冲突（语料里互相矛盾）→ 混；③ 上下文违背（有 RAG 依据却答非所问）→ 上下文幻觉（Lookback Lens 研究的就是这类）。

6.2 检测的两族谱：黑盒一致性 vs 白盒探针

表方法	思想
semantic Entropy（Farquhar et al., Nature 630, 2024）；SelfCheckGPT（EMNLP 2023, 2303.14451）	采样多次，看模型**自己跟自己是否一致**；不一致 = 大概
ookback Lens（2407.07071）；SE Probes（2406.15927）	直接读**注意力图/隐状态**训练一个小分类器预测幻觉

- 语义熵（SE）：把 n 个采样答案按语义聚成簇，SE = 簇分布的熵——“北京、北京、北京”→ SE=0；“1642米、6385米、375米…”→ SE 接近 $\ln(n)$ 。它不需要标准答案，是“无参考幻觉检测”，Nature 2024 那篇的卖点。
- SelfCheckGPT 更朴素：拿一个（更强的）模型判断 n 个答案互相之间是否支持，一致性打分即幻觉分数——和 SE 同思想，只是“聚类器”换成了 LLM。
- SE Probes 的工程价值：发现隐状态里就藏着 SE 的信息——训一个线性探针预测 SE，把“采样 n 次 + 聚类”压缩成“一次前向”。Lookback Lens 则发现上下文幻觉会写在“回看检索原文”与“看新写内容”的注意力比例里，同样一次前向可读出。

实验 A 实测（脚本 11 的简化版 SE：表层规整 → 0.5B mean pooling 句向量 → 批内中心化 → 余弦 ≥ 0.8 聚类；原论文用 NLI 双向蕴含聚类）：

30 题汇总：多数答案错误 16/30 题
SE 均值：多数答案正确 0.836 nat vs 错误 1.713 nat
AUROC(SE → 预测多数答案错误) = 0.864

> 复跑提示：AUROC/温度表这类采样型指标即使固定种子（seed=1337），换 GPU/环境

> 重跑仍会小幅漂移 (n=30 的统计噪声很宽：换卡实测 AUROC 从 0.71 到 0.86 都出现过)。
> 看趋势不看个位点——SE 在"错误"题上系统性更高、AUROC 明显大于 0.5，这才是结论。

真实采样长什么样 (同一题的 10 个答案；样例取自另一次运行，与上方汇总表的 seed 相同但 GPU 不同——采样漂移的直观展示)：

Q: 贝加尔湖最深处大约是多少米？(SE=2.16, 9 个语义簇)
1,647米 / 2791米。 / 4125米 / 贝加尔湖最深处大约有6385米。 / ...
Q: 中国的首都是哪座城市？(SE=0.00, 1 个语义簇)
北京 / 北京 / 北京。 / 北京 / ...

> △ 踩坑实录 (写进了脚本注释)：mean pooling 句向量挤在窄锥里——实测"北京"vs
> "上海"的原始余弦高达 0.95+，任何阈值都分不开。解法是批内中心化 (减去这 10 个
> 答案的均值再归一化)：相同字符串恢复到 1.0、不同答案落到 ≤0.3，阈值 0.8 才可用。
> 这是各向异性 (anisotropy) 问题的最小现场版。

6.3 破除迷思："把温度调低"不是事实性方案

直觉说 T→0 更"保守"更"事实"。实验 B (温度 sweep, 多数投票答案对错为标签)：

T	易幻觉 trivia (15 题)	稳定事实 (15 题)	总体幻觉率
0.0	80.0%	26.7%	53.3%
0.3	80.0%	33.3%	56.7%
0.7	80.0%	26.7%	53.3%
1.0	80.0%	26.7%	53.3%

曲线几乎平直 (见脚本目录 [output_hallucination.png](#)) ——与 Renze 2024 (2402.05201, 温度对 LLM 解题表现的影响任务相关、普遍不显著) 一致。机理很直白：
知识没存进权重里，把采样调"保守"也编不出正确答案。低温只消灭"多样性"，不生产"知识"。

6.4 校准：对齐的隐性代价 + 我们的复现

GPT-4 技术报告 (2303.08774) 给过一张著名对比图：预训练模型校准曲线贴对角线，
RLHF 之后明显劣化 (更自信，但自信不等于更对) ——对齐训练压扁了概率分布的信息量。
注意这是 GPT-4 时代 (RLHF/PPO 系) 的结论；对齐后真实性/弃权训练方向的新证据见
TruthRL (2509.25760, RL 框架，非 DPO 校准专论)。

实验 C (base vs Instruct, 同一批 20 道 3 选 1, 选项 token 受限 softmax 为置信度, ECE 10 桶)：

模型 准确率 平均置信度 ECE(10桶)
Qwen/Qwen2.5-0.5B (base) 95.0% 78.2% 0.171
Qwen/Qwen2.5-0.5B-Instruct (对齐后) 65.0% 79.2% 0.198

> △ 诚实解读：我们的玩具复现里 ECE 从 0.171 升到 0.198——方向与 GPT-4 报告一致
> (对齐后校准变差)，但量级远不及原报告；更稳定的信号是偏差方向翻转——base 欠自信
> (95% 对 vs 78% 置信)，Instruct 过自信 (65% 对 vs 79% 置信)：置信度原地不动而正确率
> 大幅下降，即"相对过自信"的校准恶化现场。0.5B + 20 题样本量，**看方法、看方向，
> 别看效应量**。

6.5 缓解的工程正道

1. RAG：生成前检索外部依据拼进上下文，把"闭卷考试"变"开卷考试"——治"知识缺失"型幻觉（全链路实战见 [Part 18](../Part18_rag/tutorial/README.md)）。
2. 弃权训练（abstention）：教模型说"我不知道"——把"拒答"也纳入奖励，而不是硬答。
3. 采样一致性过滤：用 SE/SelfCheckGPT 分数做低置信路由（转人工/转检索）。
4. △ 别指望温度：见 6.3，那是采样参数不是知识来源。

<details>

<summary>Q1: 语义熵检测幻觉，为什么不需要标准答案？什么时候会失手？</summary>

A: SE 只用"同一题多次采样的语义一致性"这一个信号——模型参数里知识稳定时，采样会收敛到同一答案（SE→0），知识缺失时分布平坦、每次编一个（SE→ln n）。失手场景：① 模型稳定地错（高置信的错误知识，采样全一致）——SE 检不出来，这是它和"对答案"式评估的本质差异；② 答案空间天然多模态（"举三个例子"类开放题），一致性低但不算幻觉；③ 需要"语义"聚类器足够好（我们的余弦版比 NLI 版弱）。

</details>

<details>

<summary>Q2: 为什么 RLHF 会损害校准？怎么补？</summary>

A: 机理：偏好优化鼓励"确定性输出"（评审偏爱果断答案）→ 概率分布被压扁，输出概率不再反映真实的不确定性；GPT-4 报告的校准曲线劣化就是这个代价。补救：① 训练后做温度缩放/Platt scaling 重校准（便宜，最常用）；② 奖励里显式加入校准项（答对时奖励高置信、答错时奖励低置信）；③ 弃权训练让不确定性有出口。对齐算法族在校准上的差异（含 DPO 系）尚在研究；真实性/弃权方向的最新工作见 TruthRL (2509.25760)。

</details>

<details>

<summary>Q3: Lookback Lens / SE Probes 这类白盒方法相对黑盒采样的优劣？</summary>

A: 优：一次前向就能打分（黑盒要 n 次采样），延迟和成本差 n 倍，可上线实时用。
劣：① 探针要额外训练且跨模型/跨域泛化存疑；② 依赖内部访问权（API 模型拿不到隐状态/注意力）；③ 可解释性差——分数高不告诉你"哪句话是编的"。工程上常见组合：白盒探针做线上实时过滤，黑盒一致性做离线评测金标准。

</details>

7. 安全对齐：拒绝行为、攻防与评测（方法论视角）

- > 本节配套脚本 11 的 可选段。△ 出于安全考虑，**本节与脚本只讲方法本身，> 不罗列任何具体的违规/有害提示语样本**——涉及真实攻防数据的实验，数据由读者按> 论文附录自备（防御性视角）。

7.1 拒绝行为的机理：一个方向

单方向假说（Arditi et al., 2406.11717）：LLM 的拒绝行为集中体现在残差流的一个方向 r 上——把 r 从所有表征中投影掉（方向消融），模型"不会拒绝"；反过来沿 r 增强，模型对无害请求也拒绝。这意味着对齐训练学到的安全行为比想象的线性得多、也脆弱得多。

论文的三件套（脚本 11 的 `refusal_direction()` 按此实现）：

- ① 单方向假说：拒绝 ≠ 分散在千万参数里，而是集中在一个方向

- ② diff-in-means : $r = \text{mean}(h_{\text{拒绝类提示}}) - \text{mean}(h_{\text{普通提示}})$
(h = 提示最后一个 token 的中层隐状态；与 activation patching 提取的方向几乎重合，但便宜一个数量级)
③ 方向消融 : $h \leftarrow h - (h \cdot \hat{r})\hat{r}$ ；权重级等价于对每个 Linear 做 $W \leftarrow W - \hat{r} \hat{r}^T W$

论文附录 C 还给了一套方向选择协议：对每个候选层提取方向后，按三个分数挑选最优层——bypass 分数（消融后在违规请求集上的拒绝指标，衡量"拒绝被解除的彻底度"，取最小）、induce 分数（把方向加法到正常请求上，看能否诱导出拒绝，验证方向本身有效）、KL 散度（消融前后在正常请求上的输出分布偏移，须足够小——这才是"正常请求不被误伤"的条件）——严格说 bypass 是被最小化的优化目标、induce>0 与 KL<0.1 是过滤阈值、层搜索限定 $l<0.8L$ （论文附录 C.1）。diff-in-means 只是提取器，选哪一层的方向才是效果差异的大头。

> △ 这个发现的正确读法是双向的：它既解释了拒绝行为"是怎么存储的"（机理），
> 也演示了"删掉一个方向就能卸掉拒绝"（脆弱性）。防御者的结论不是"藏好这个方向"，
> 而是安全不能只靠模型内化——要配合外部分类器与评测（见 7.2、7.3）。

脚本 11 的呈现方式：`refusal_direction(model_path, prompts_file)` 从读者自备的 jsonl 文件（每行 `{"text": ..., "label": "refusal"/"normal"}`，normal 组即普通问答语句）读取两组提示 → diff-in-means → 报告方向上投影的 AUROC 与消融前后对比；数据文件缺失或 7B 模型未就绪时打印说明跳过（rc=0）。**实验数据需读者按论文附录自备，7B 级模型效果最好。**

7.2 攻防一句话版

- 攻（红队）：GCG（Zou et al., 2307.15043）思想一句话——把"让模型答出违规内容"形式化为对对抗后缀字符串的梯度优化（离散搜索），且优化出的后缀在黑盒模型间可迁移。此后所有"越狱"研究基本都沿"可优化的攻击面"这个框架展开。
- 防（工业界）：Constitutional Classifiers（Anthropic, 2501.18837）——用 AI 宪法（成文原则）合成海量训练数据，训练输入/输出双向分类器包在模型外层：拦截率高、误拒率可控、推理开销个位数百分比。思路与模型内化的 RLHF 互补：外挂的、可独立迭代的护栏。

7.3 评测：用什么基准说话

基准	定位	现状
HarmBench** (2402.04249)	标准化对抗攻击评测套件（统一攻击方法 × 防御 × 判定器）	当前主证据
JailbreakBench**	开源越狱 artifact + 公共排行榜（可复现、可对比）	当前主证据
AdvBench	早期的有害请求示例集	△ **已过时，不作主证据**（规模小、被后续工作过拟合）
TruthfulQA	2021 年的误导性问题集	△ **已过时**（被 leaderboard 刷分吸收，区分度不足）

> 面试上一句"我们用 HarmBench + JailbreakBench 做红队回归，AdvBench 只做烟雾测试"
> 就能暴露出你是看过 2024+ 文献的人。

<details>
<summary>Q1: 为什么"拒绝集中在单个方向"会被视为对齐脆弱性的证据？</summary>
A: 它说明安全行为在表征空间里是低维、可定位、可外科手术式移除的：一个方向消融操作（不动任何训练）就能让 RLHF 训出来的拒绝大幅失效。推论：① 对齐内化不等于鲁棒，微调或表征层面的扰动可能卸掉它；② 防御要分层——内化之外还有外部分类器、输出扫描、权限控制。注意单方向假说在更大/更新的模型上是否仍然成立是开放问题。
</details>

<details>
<summary>Q2: Constitutional Classifiers 相比"再跑一轮 RLHF 加固"的优势？</summary>
A: ① 数据由宪法+合成管线生成，不用人工标注敏感数据（合规与隐私都好办）；
② 外挂分类器可独立迭代、灰度、回滚，不动模型本体——出问题时不用重训模型；
③ 可解释：拦不拦得起一条请求能对应到宪法条款。代价：误拒率需要调（拦得狠会伤正常可用性），推理增加一路分类器开销（论文报告约个位数百分比）。
</details>

<details>
<summary>Q3: 为什么 AdvBench / TruthfulQA 不再适合当主证据？</summary>
A: AdvBench 只有几百条静态示例，且公开太久——很多"防御"是在它的已知分布上过拟合，新攻击一换数据就失效；TruthfulQA 是 2021 年为小模型设计的误导性问答，题目已被各大 leaderboard 反复刷分，区分度不足。现代做法：HarmBench（标准化攻防套件，攻击方法可控可比）+ JailbreakBench（公开 artifact、排行榜防"自说自话"）。
</details>

8. 中国合规一页纸（生成式 AI 上线四件套）

面向国内部署的"什么时候需要什么"，一页说清（条款号按现行规定）：

	要点
算法推荐管理规定》+《生成式 AI 服务管理暂行办法》第 17 条	具有舆论属性或社会动员能力的服务，上线前履行算法备案
深度合成管理规定》	深度合成服务提供者和技术支持者都要备案（与算法备案并行，故称"双备案制"）
一套实操文件	基于已备案模型提供服务的，登记申明即可（"登记"流程出自配套实操文件，第 17 条本
智能服务安全基本要求》	上线前过安全评估：违法有害内容、歧视、商业秘密侵犯、知识产权、伦理等维度自测
生成内容标识办法》（**2025-09-01 施行**）	AI 生成内容要加**显式标识**（用户可见）+**隐式标识**（元数据/水印），传播链路

面试答法（四件套）："算法备案 + 大模型备案/登记 + 上线前安全评估题库自测留痕 + AI 生成内容显式/隐式标识"——四件套说完，再补一句"合规是产品能力：备案材料里的语料安全、模型安全、安全措施三块，正好对应我们训练侧的数据治理和本节的评测体系"。

> 合规与技术不是两张皮：安全评估题库本质上就是一套**规则评估 + LLM-judge 混合的
> 红队回归测试**（本章 § 1 的三范式直接适用）；内容标识的隐式水印则与 Part 8 的
> tokenizer/采样知识同源。标识办法自 2025-09-01 施行已满一年（写作时点），已是从业常识。

<details>
<summary>Q1: 公司用别人已备案的大模型 API 套壳做产品，还要自己备案吗？</summary>
A: 要走登记而非完整备案：基于已备案模型提供服务的，按暂行办法第 17 条申明其使用的模型即可；但若做了显著修改（继续预训练/微调改变能力边界）则按自研模型对待，需要自己做安全评估。算法备案（如有舆论属性/社会动员能力）与内容标识义务不豁免。
</details>

<details>
<summary>Q2: "显式标识"和"隐式标识"各防什么？</summary>
A: 显式标识（界面提示、文本角标等）防人被误导——用户知道这是 AI 生成的；
隐式标识（文件元数据、数字水印）防机器/传播链路——转发、二次加工、平台审核时仍可溯源。两者缺一防：只显式易被截图抹掉，只隐式用户当场无感知。
</details>

<details>
<summary>Q3: 安全评估题库和我们本章的评估体系是什么关系？</summary>

A: 同一套方法论：题库 = 固定种子、固定题集的规则评估 + LLM-as-judge 判分；上线后持续监测 = 防"benchmark 污染"式的自测泄漏（题库要轮换）。区别只在维度表换成了监管要求的违法有害、歧视、知识产权等条目——所以说"合规是产品能力"，不是文书工作。

</details>

面试直通车

<details>
<summary>Q1: 训练数据里混进了 benchmark 原题，模型分数虚高，怎么发现？</summary>

A: 三条路：① 训练语料与测试集做 n-gram (13/20-gram) 重叠扫描，命中即污染；
② 行为探测——让模型补全测试题干，能一字不差复述答案的基本是背的；
③ 镜像集对照——GSM1k 思路：同分布新题重测，掉分幅度暴露过拟合程度。

</details>

<details>
<summary>Q2: LLM-as-judge 有哪些系统性偏差？怎么修？</summary>

A: 位置偏差（先出现的占优→交换 A/B 重判）、长度偏差（长答案占优→length-controlled win-rate）、自偏好（评审模型偏爱自己家族的输出→换多个 judge 交叉）。MT-Bench 的一致性数据（85% vs 人类 81%）说明可用，但要带这些修正。

</details>

<details>
<summary>Q3: 两个模型 tokenizer 不同，A 的 ppl=3.2、B 的 ppl=8.1，谁好？</summary>

A: 不可直接比。ppl=每 token 平均交叉熵的指数，token 越碎（词表越大）单 token 越好预测，ppl 天然越低。可比做法：同一份评测语料换算 bits-per-byte（总 NLL / 总字节数再转 2 底指数），或者干脆用同一批下游任务分数比。

</details>

> 延伸对照（LLMs-from-scratch）：rasbt ch07 的 [ollama_evaluate.py](#) 是 LLM-as-judge 的最小可运行实现（本地 Ollama 评审模型打分）——想动手验证本节"LLM-as-judge 偏差"，从它开始。

课程总结：你现在的位置

Part 1-6 会训练一个语言模型 → loss 下降曲线

Part 7-8 会走完现代 LLM 全流程 → 一个能对话的模型

Part 9 知道它跑在什么硬件上 → 优化方向的判断力

06 章 知道怎么让它更快更省 → 上线能力

07 章 知道怎么证明它真的更好 → 科研与工程的可信度

07 章下半 知道它何时在瞎编、怎么算安全合规 → 幻觉检测（SE）、校准（ECE）、安全评测（HarmBench）、合规四件套

下一步（课程路线图 docs/course_roadmap_v2.md）：Part 10 分布式训练——单卡装不下模型的那天。

[< 上一章：推理与服务](06_inference_and_serving.md) | [Part 8 README](README.md)

08_lora_and_classification

08 — LoRA 与分类微调：参数高效微调从零写

- > SFT 要更新全部参数，但真实世界里最常见的诉求是"在一张卡上、不破坏预训练能力、快速把模型调到我的任务上"——这就是 参数高效微调（PEFT），其中事实标准是 LoRA。
- > 本章从零实现它（对照 rasbt/LLMs-from-scratch 附录 E），顺带走一遍分类微调的标准范式
- > （rasbt ch06）——这是很多从零课程漏掉、但工作中天天用的技能。

前置知识

- 02 章：SFT 与 prompt masking（LoRA 就是在它之上做减法）
- 10 章（Part 10）：显存账本 16 字节/参数——本章算"LoRA 省多少"直接用

1. 问题：全参微调贵在哪

用 Part 10 的账本算 7B 模型全参微调：可训练参数 12 字节/个（fp32 参数+梯度+AdamW 两个状态）
→ $7B \times 12 = 84GB$ 起步，再加激活。这就是"全参微调 7B 要多卡"的全部原因。

LoRA（Low-Rank Adaptation, Hu et al. 2021）的洞察：

微调引起的权重变化 ΔW 是低秩的（任务适配不需要全部 7B 个自由度）

→ 不学 ΔW 本身，学它的低秩分解 $\Delta W = B \cdot A$ （ $B: d \times r, A: r \times k, r \ll d, k$ ）

→ 冻结 W ，只训 $A、B$ ：可训练参数从 $d \times k$ 降到 $r \times (d+k)$

- 三个实现细节决定成败：① A 高斯初始化（ $1/\sqrt{r}$ 缩放）、 B 初始化为 0
→ 训练起点 $\Delta W=0$ ，不破坏预训练表征；② 前向 $y = Wx + (\alpha/r) \cdot B(Ax)$ ， α/r 缩放让调 r 时学习率尺度稳定；③ 推理时可把 BA 合并回 W （ $W += (\alpha/r) \cdot BA$ ），零额外延迟。
- $\triangle$ LoRA 省的是优化器状态+梯度的显存（可训练参数从 100% → 百分之几），权重本体（bf16 推理副本）仍然全量在显存里——"LoRA 微调 70B 单卡可行"靠的是 4-bit 量化底座（QLoRA，见 Part 12）。

2. 从零实现（跑 `scripts/10_lora_from_scratch.py`）

```
`python
class LoRALinear(nn.Module):
    def __init__(self, linear: nn.Linear, r=4, alpha=8.0):
        super().__init__()
        self.linear = linear
        for p in self.linear.parameters():
            p.requires_grad_(False) # 冻结 W
        out_f, in_f = linear.weight.shape
        self.A = nn.Parameter(torch.randn(r, in_f) / math.sqrt(r))
        self.B = nn.Parameter(torch.zeros(out_f, r)) # 零初始化 → 起点无损

    def forward(self, x):
        return self.linear(x) + (self.alpha / self.r) * (x @ self.A.T) @ self.B.T`
```

注入后只让 BA + 分类头可训练（其余全部 `requires_grad_(False)`），然后按 rasbt ch06 的分类微调范式跑对比——任务：序列含 ≥ 3 个 '7' $\rightarrow$ 正类（玩具版垃圾邮件识别）：

全参微调: 可训练 230,226 参数 (100%) | 验证 acc = 0.955

LoRA 微调: 可训练 7,872 参数 (3.4%) | 验证 acc = 0.924 $\leftarrow$ 注入 4 层, $r=4$

训练显存估算: 全参 2.8 MB vs LoRA 0.5 MB（真实 7B 上是 GB 级 vs MB 级）

- 3.4% 的参数达到接近的精度——"低秩足够"假设的最小可验证证据。
- 把数据阈值从 3 调成 4 增加任务难度，会看到 LoRA 与全参的差距开始拉开（ r 太小 $\rightarrow$ 提高 r ）。
- $\triangle$ 实现坑（我们真实踩到）：注入新建的 A/B 默认在 CPU，注入后要再 `.to(device)` 一次；分类标签要 `.long()` 才能进 `cross_entropy`。

3. 分类微调范式（rasbt ch06 的标准流程，4 步）

- ① 换头：`lm_head` $\rightarrow$ `cls_head` (`Linear(hidden, n_classes)`)，隐藏向量取【最后一个位置】
- ② 数据：(sequence, label) 对；不需要 prompt/masking —— 这是它与 SFT 的本质区别
- ③ 冻结策略任选：全参 / 只训头 / +LoRA —— 三档本脚本都给了骨架
- ④ 评测：accuracy（不是 ppl）—— 语言模型的评估指标在分类任务上不适用

- 什么时候用分类微调而不是 SFT？—— 输出是离散类别（情感/风控/路由/质检）时。工作里"用 LLM 做分类器"比"用 LLM 生成"更常见于生产管线，这是 rasbt ch06 值得单独一章的原因。

4. 与 Part 12 的关系

本章 = 概念与从零实现（几十行，看得见每一行）；Part 12 = 工业工具（LLaMA-Factory 的 LoRA/QLoRA/DPO 全家桶 + 7B 模型实战）。学完本章再去 Part 12，工具的每个 yml 字段（`lora_rank`/`lora_alpha`/`lora_target`）你都知道对应哪行代码。

学完本部分你能...

- 手写 LoRALinear，说清 A/B 初始化约定与 α/r 缩放的作用
- 算清 LoRA 省的是哪部分显存、不省哪部分
- 走通分类微调 4 步范式，说出它与 SFT 的本质区别
- 用"可训练参数比例 + acc 对比"验证参数高效微调的有效性

课后练习

<details>

<summary>Q1: 为什么 B 初始化为 0 而 A 不为 0？两个都为 0 行不行？</summary>

A: $B=0$ 使初始 $\Delta W=BA=0$ ，训练起点等价原模型。若 A 也为 0，则 $\partial L / \partial A \propto B=0$ 、 $\partial L / \partial B \propto A=0$ ，两个矩阵的梯度都恒为 0 —— 永远学不动。所以必须"一零一非零"打破对称。

</details>

<details>

<summary>Q2: LoRA 应该注入哪些层？注入 attention 还是 MLP？</summary>

A: 原论文在 attention 的 W_q/W_v 上实验；实践共识（和 QLoRA 论文）是"全部 Linear 都注入

效果最好", 预算有限时优先 attention 的 q/v。注入层越多可训练参数越多、越接近全参效果。
本脚本注入 MLP 两个 Linear 是为了 toy 上快速演示, 把 target 换成 attention 可自行实验。

</details>

<details>

<summary>Q3: 推理时 LoRA 有额外开销吗? 多租户 (一个基座 + N 个适配器) 怎么办? </summary>

A: 合并回 W 后零开销; 不合并则有 BA 的额外矩阵乘。多租户场景 (vLLM 的 multi-LoRA) :
基座共享一份, N 个适配器按请求动态切换——这正是 Part 12/14 工具链的卖点之一。

</details>

课后作业

跑通 [scripts/10_lora_from_scratch.py](../scripts/10_lora_from_scratch.py) 并完成三个实验：
r=2/4/16 对比 acc、注入 attention vs MLP 对比、任务难度 (阈值 3→4) 对 LoRA-全参差距的影响。

下一步

参数高效微调的"从零"到此为止。工具级实战 (QLoRA 7B、DPO-LoRA、WebUI) 见 Part 12 ;
工业 RL 框架见 Part 11。

[Part 12 LLaMA-Factory 微调实战 (拟开)](../Part12_finetune_llamafactory/tutorial/README.md)

[<- 上一章：评估学](07_evaluation.md) | [Part 8 README](README.md)

09_reasoning_models

09 — 推理模型与 test-time compute

- > DeepSeek-R1 的核心创新是把 RL 用在"思维链生成"上, 让模型在 <think> 段自主推理。
- > 本章手写这条管线的最小闭环 (SFT 格式 → GRPO 推理 RL → test-time compute) ,
- > 实测 self-consistency 的"算力换准确率"效应。
- > 跑 [scripts/09_reasoning_models.py](../scripts/09_reasoning_models.py)。

前置知识

- Part 8 04 章：GRPO 组内优势 (本章的 RL 算法基础)
- Part 8 07 章：评估学的"规则奖励"思想 (RLVR = 用可验证规则当奖励)

1. DeepSeek-R1 的四阶段训练管线

阶段	做法	解决什么
R1-Zero	纯 RL (GRPO) , 无 SFT	证明"纯 RL 可涌现推理"——但可读性差、语言混杂
R1 cold start	少量长 CoT SFT	给 RL 一个"会说话"的起点

阶段	做法	解决什么
R1 推理 RL	GRPO on 推理任务（准确率+语言一致性奖励）	提升推理能力
R1 全场景	拒绝采样 SFT（60w 推理 + 20w 通用）+ RL	兼顾有用性与无害性

- 奖励 = 规则准确率 + 格式分（非神经网络 RM，防 reward hacking）。准确率 = 答案是否正确（数学可验证）；格式 = `<think>` 段是否存在且在 `<answer>` 之前。

2. 手写两阶段（脚本 09 的核心流程）

```
`python
阶段 1: cold start SFT ——
教模型"说格式"（<think>步骤</think>
<answer>答案</answer>）
```

阶段 2: 推理 RL（GRPO 简化版）—— 奖励 = 结果正确 + 0.5 × 格式分

```
`
实测（5000 SFT + 300 GRPO，单位数加法）：
SFT loss → 0.2177（模型学会了 think+answer 格式与正确答案）
```

3. Test-time compute：算力换准确率

self-consistency（Wang et al. 2022）：同一 prompt 采样 N 条轨迹，取众数答案。为什么有效：正确答案通常比错误答案更"一致"（多次采样更可能命中）。

```
`
n=1: 准确率 56% ← 单次采样
n=4: 准确率 46% ← 玩具模型波动（n=4 时众数不稳定）
n=8: 准确率 58% ← 更多样本 → 众数趋近真实分布 → 恢复并超过 n=1
```

- ⚠ 诚实解读：玩具模型的 n=4 比 n=1 低是采样噪声——真实模型上 self-consistency 单调提升更明显（任务越难、模型越好，提升越大）。核心主张是：**n ↑ → 众数趋近条件分布的众数 → 正确答案被"voted up"***。

4. 脚本 09 的三个实现细节（面试向）

1. 答案提取：`<answer>` token 后的第一个数字 token。真实系统用更鲁棒的抽取（如 GSM8K 的 ##### 42 格式 + 正则最后一数字）。
2. 温度：RL 采样 1.1（保持多样性）、self-consistency 1.1（保持多样性以让众数有意义）——温度=0 时退化为贪心解码，self-consistency 退化为单次贪心。
3. RL 奖励的组合：accuracy（结果对错）+ 0.5 × format（think/answer 段结构）。

格式奖励保证模型"先想再答", 准确率奖励驱动推理质量。

学完本部分你能...

- 画出 R1 四阶段管线, 说出每阶段的输入/输出/奖励
- 实现 self-consistency 并实测" $n \uparrow \rightarrow \text{准确率} \uparrow$ "
- 解释 format reward 为什么必要 (防止模型跳过 think 段直接给答案)
- 区分 test-time compute 与训练时 scaling (前者是推理时算力, 后者是预训练算力)

课后练习

<details>

<summary>Q1: R1-Zero 和 R1 的核心区别? 为什么 R1-Zero 不够? </summary>

A: R1-Zero = 纯 RL 无 cold start。它能涌现推理 (aha moment) 但输出可读性差、语言混杂 (中英夹杂)。R1 加了 cold start SFT (少量长 CoT 数据教模型"好好说话"), 后续再做多阶段 RL。核心教训: 纯 RL 能变聪明, 但不会自动变得"好读"。

</details>

<details>

<summary>Q2: 为什么 RL 阶段用规则奖励而非神经网络奖励模型? </summary>

A: 数学/代码类任务可以机器验证 (规则不可被 hack); NN-RM 有被 reward hacking 的风险 (模型学会利用 RM 的盲区而非真正变强)。R1 论文明确说用规则奖励避免此问题。

</details>

<details>

<summary>Q3: self-consistency 和 best-of-N 有什么区别? 各适合什么场景? </summary>

A: self-consistency = 采 N 条取众数 (适合有明确答案的任务); best-of-N = 采 N 条用验证器/奖励模型选最佳 (适合可打分但答案不唯一的任务)。前者更简单、后者更灵活。本质上都是"用 N 倍推理算力提升准确率"。

</details>

课后作业

无独立作业——本脚本的 test-time compute 实验即作业 (Part 16 02 章的 img2img/ControlNet 作业使用 Assignment 16)。

下一步

多模态如何让模型"看懂"图片? → Part 15 多模态理解。

[Part 15 多模态理解](../Part15_vision_language/tutorial/README.md)